

Otter: Scalable Sharding-Based Atomic Broadcast with Abortable Fork Detection

Xin Wang
wangxin87@tsinghua.edu.cn
Tsinghua University
China

Xiao Sui
suixiao@mail.tsinghua.edu.cn
Tsinghua University
China

Sisi Duan^{*†}
duansisi@tsinghua.edu.cn
Tsinghua University
China

ABSTRACT

Sharding is a generic approach to enhance the scalability of distributed systems. In recent years, many efforts have been made to scale the consensus mechanism of blockchains from sharding. A crucial research question is how to achieve the sweet spot of having a relatively small shard size (to achieve decent performance) while achieving an overwhelming probability of correctness (so the system is safe and live). Many recent works fall into the two-layer design that uses some *coordinating shards* to monitor the correctness of other shards (CCS 2022, NDSS 2024, INFOCOM 2023). All of them involve expensive communication costs between the shards, significantly degrading performance.

We present Otter, a scalable partially synchronous sharding-based Byzantine fault-tolerant atomic broadcast (ABC) protocol. We use coordinating shards in a completely new way. In particular, we randomly sample coordinating shards to directly *participate* in the consensus protocol. Such a random sampling mechanism makes it possible to analyze the correctness of the ABC protocol using a probabilistic model. In this way, we can significantly lower the shard size (informally, from over 1,200 in previous work to around 100) without lowering the probability of correctness. We also present a new notion called abortable fork detection (AFD) that might be of independent interest. Our evaluation results on Amazon EC2 using up to 1,000 replicas show that Otter achieves up to 4.38x the throughput of the state-of-the-art protocol.

KEYWORDS

atomic broadcast, sharding-based protocol, blockchains, Byzantine fault tolerance

1 INTRODUCTION

Current blockchain systems are known to suffer from scalability challenges, where the performance degrades significantly as the system size grows. In recent years, sharding, a generic approach to enhance the scalability of the system, has been broadly studied for blockchain consensus protocols [3, 6, 20, 30, 31, 34, 36, 37, 48, 50, 54]. In the context of sharding-based consensus protocol, the idea is to divide the replicas into smaller groups called shards and parallelize the consensus process.

Many earlier works assume that each shard is *correct* and does not have more than one-third or half faulty replicas [6, 37, 42, 54]. This might be an overly strong assumption. For instance, in a partially synchronous network (where the message processing and

transmission delay is bounded by an unknown Δ), consider a system with n replicas, among which at most f are Byzantine faulty. It is widely known that $n \geq 3f + 1$ is optimal for the system to be *correct* (i.e., *safe* and *live* [13]). In such a case, even if we randomly sample the shard members, the probability that some shards are faulty (with more than one-third of faulty replicas) cannot be negligible. As a solution, several sharding-based approaches sacrifice optimal resilience to ensure the correctness of the shards. For instance, Omniledger [34] assumes $n \geq 4f + 1$, and each shard has no more than one-third of faulty replicas in a partially synchronous network. Reticulum [50] assumes $n \geq 3f + 1$, and each shard has no more than half faulty replicas in a synchronous network (where Δ is known).

In fact, the observation above is aligned with the committee sampling-based approaches for Byzantine fault-tolerant consensus (BFT for short) [2, 8, 9, 17, 18, 28, 33, 47]. Namely, consider $f < (1/3 - \epsilon)n$, where $\epsilon \in (0, 1/3)$. We sample a committee of λ size. The probability that there are at most one-third faulty replicas (so the correctness of the system will not be violated) can be $1 - \text{negl}(\kappa)$, where κ is the security parameter¹. In practice, λ has to be large enough for the probability to be close to 1. For instance, to make the probability of achieving safety and liveness $1 - 10^{-9}$ and to set $\epsilon = 0.12$, a committee size of at least 2,000 replicas is needed [28, Figure 3]. The same applies to sharding-based protocols. Suppose we use conventional BFT protocols [11, 16, 22, 52] at the shard level. The protocol becomes unusable for the shard size to be over 2,000, as most BFT protocols achieve decent performance only when the number of replicas is below 100.

The sweet spot of decent performance and overwhelming probability of correctness. One natural observation is that if the shard size is smaller, the performance of the system can be improved. Many recent sharding-based approaches exploit this insight. For instance, GearBox [20] proposes a beautiful result that separates the thresholds for safety and liveness (that is somehow aligned with known results such as multi-threshold BFT [39]). The idea is to use a small shard size so there might be more than one-third of faulty replicas. By increasing the threshold for the *quorum size* (a Byzantine quorum is typically $2f + 1$ out of $n = 3f + 1$ replicas in a partially synchronous network [13]), safety will never be violated, i.e., there will not be hard forks of the chain. Of course, liveness cannot be ensured if the shard has over one-third of faulty replicas. GearBox thus uses another *layer* of shards called committees to monitor the liveness of the shards. The shard is reconfigured (e.g., by increasing the shard size) if some shard does not produce any new

[†] Corresponding author.

^{*} Sisi is also with State Key Laboratory of Cryptography and Digital Economy Security, Zhongguancun Laboratory, and Shandong Institute of Blockchains.

¹ Here, we consider λ a security parameter as well. We intentionally differentiate it from κ where κ usually denotes the size of the signatures or hashes.

protocol	timing	$f <$	comm. (entry shard)	comm. (coordinating shard)	fork detection
GearBox* [20]	partial sync.	$(1/3 - \epsilon)n$	$O(L\lambda + \kappa\lambda^2 + n)$	$O(L\lambda^2 + \kappa\lambda^3 + \kappa n)$	✗
Reticulum* [50]	sync.	$n/3$	$O(L\lambda + \kappa\lambda^3)$	$O(\kappa\lambda^3 + L\lambda)$	✗
CoChain [36]	partial sync.	$(1/3 - \epsilon)n$	$\frac{2}{3}O(C_\lambda^L + \kappa^2\lambda^3)$	$O(C_\lambda^\kappa + \kappa^2\lambda^3 + \kappa^3\lambda^2)$	✗
Otter (this work)	partial sync.	$(1/3 - \epsilon)n$	$O(L\lambda + \kappa\lambda^2 + \kappa\lambda^3)$	$O(\kappa\lambda)$	✓

Table 1: Overview of sharding-based protocol from using coordinating shards. None of the previous works analyze their complexities, so we provide a detailed analysis of them in Appendix A. λ is the number of replicas in each shard. κ is the length of the security parameter. L is the length of each block. ϵ is a small constant. *The coordinating shard for GearBox and Reticulum is called the *committee* and the *control shard*, respectively. For all the protocols considered in this table, we assume the standard PKI assumption where a set of digital signatures (e.g., ECDSA) are used as a quorum certificate. The highlighted complexity in blue is the cost for shard management. $\frac{2}{3}O(C_x^y)$ denotes the communication complexity of a conventional BFT protocol among x replicas for y -bit inputs.

blocks. Several follow-up works further extend this result [50, 56]. These approaches generally fall into the two-layer design, using a layer with smaller shards for high performance and a layer with larger *coordinating* shards as a “fallback” to achieve the security goals of the system. We summarize them in Table 1.

Our insight is that the current two-layer design with different shard sizes has yet to achieve the sweet spot of decent performance and overwhelming probability of ensuring the correctness of the system. Take GearBox as an example; if a shard has more than one-third of faulty replicas, liveness cannot be guaranteed. One still has to have a large shard size to achieve liveness, and reconfiguration of the shards may introduce unavoidably high communication costs. A similar observation has been made recently by Reticulum [50]. The solution by Reticulum is to use a large enough *control shard* as the coordinating shard. The control shard participates in the operation of a shard to ensure the correctness of the system. To do so, Reticulum sacrifices optimal resilience, and the communication cost for the coordinating shard is also high (see Table 1).

Thus, an open research question is:

Can we achieve the sweet spot of having a moderate shard size (to achieve decent performance) and an overwhelming probability of correctness (so the system is safe and live)?

Our contributions. We present Otter, a practical sharding-based Byzantine fault-tolerant atomic broadcast (ABC) protocol² that achieves near-optimal resilience in the partially synchronous model, i.e., $f < (1/3 - \epsilon)n$. Otter supports a moderate shard size that makes the system practical enough and meanwhile achieves an overwhelming probability of correctness. We propose a new *multi-shard coordinating atomic broadcast (MC-ABC)* primitive for building sharding-based ABC protocols. Our technical pathway is summarized below (also in Figure 1).

- **A practical normal-case protocol (Sec. 5).** We present a protocol that randomly samples multiple coordinating shards to participate in the agreement of *every* block. In particular, we provide a four-phase protocol where the third and the fourth phases require votes from two randomly sampled shards, respectively. Let the probability of each shard being faulty (i.e., having more than one-third of faulty replicas) be δ . Since the two coordinating shards are randomly sampled, the probability of achieving safety and liveness can be bounded by $1 - \delta^2$ (the actual proof is more

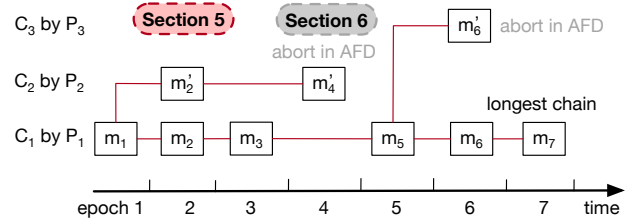


Figure 1: Overview of Otter. Each replica in a shard S *a-delivers* blocks. When the number of faulty replicas in a shard exceeds one-third of $|S|$, forks may exist (i.e., C_1 to C_3). After forks are formed, correct replicas may abort during the rotating leader protocol via our abortable fork detection (AFD) mechanism. If the number of honest replicas is $\Omega(|S|)$, correct replicas detect the forks after $O(1)$ epochs.

involved, but it exploits this insight). Jumping ahead a little bit, Table 3 shows that we only need a shard size of around 400 for the system to achieve near-optimal resilience (i.e., $f \in [\frac{1}{4}n, \frac{1}{3}n)$) and $1 - 10^{-10}$ (roughly $1 - 2^{-30}$) probability of correctness, in contrast to at least 1,200 of prior works.

- **Abortable fork detection (Sec. 6).** When we further lower the shard size, the probability of each shard being faulty increases as the shard size decreases. To make it possible to have an even smaller shard size, we provide the first formal treatment for fork detection when a shard has more than one-third faulty replicas. We propose a notion of abortable fork detection (AFD). The idea is to allow forks to exist and eventually detect them. An interesting result is that we model AFD in a similar way to the Nakamoto consensus (NC) [40]. Namely, replicas may extend their own chains, but correct replicas will either eventually converge on the *longest chain* or abort. Accordingly, our protocol in the presence of faulty shards does not achieve the conventional *finalization* property but makes finalization a probabilistic model similar to that in NC. We consider AFD a first-class primitive that might be useful for protocols beyond sharding-based ones.
- **Implementation and evaluation (Sec. 8).** We implement our protocol with more than 10k LoC and evaluate its performance on Amazon EC2 using up to 1,000 replicas. Our evaluation results show that Otter allows one to use up to 58% smaller shard size to achieve the same probability of safety and liveness. The peak throughput is up to 4.38x that of the state-of-the-art sharding-based protocol.

²ABC is only syntactically different from BFT [14]. See Sec. 3 for details.

2 RELATED WORK

Sharding-based BFT/Blockchain. Besides the work mentioned in the introduction, many works explore sharding in the Proof-of-Work model [7, 48, 53]. Additionally, many works focus on handling cross-shard transactions [5, 30, 34, 54, 56].

Faulty shard detection. The idea of faulty shard detection has been mentioned by some prior works [5, 36]. To detect faulty shards, Chainspace [5] briefly mentions that one can use a special *auditing* system that audits the logs of the replicas to detect whether some shards are faulty. The idea is that any "invalid" message from a faulty shard will not be verified by a correct shard and such a malicious behavior will be detected by the auditing system. Unfortunately, it does not specify how the auditing system can be instantiated. In addition, CoChain [36] also claims to tolerate faulty shards. Unfortunately, it does not provide any formal analysis. In contrast, we provide a formal treatment for fork detection when some shard is faulty and provide formal proof of correctness.

Slashing. A closely related concept to failure detection is slashing, first introduced by Ethereum [12]. The goal of slashing is to detect equivocation where a replica sends inconsistent proposals or votes. Such a mechanism often considers a system that does not have more than one-third of faulty replicas in a partially synchronous network or an asynchronous network. Therefore, it is different from the problem we study (where a shard may have more than one-third faulty replicas).

BFT forensics and recovery. A line of work studies BFT forensics [44, 45], which identifies the equivocating replica(s) when the system has more than f faulty replicas. It requires some party (e.g., the client) to detect the consistency of correct replicas, and then replicas can eventually *recover*. These protocols usually require the logs from *all* correct replicas to provide their logs. In contrast, we aim to model the security guarantees of a faulty group (in particular, a shard) in the presence of more than f faulty replicas. We do not require each replica to observe the logs of all correct replicas and assume a partially synchronous network.

A recent work studies how a partially synchronous BFT can *recover* after the number of faulty replicas exceeds the pre-specified threshold, i.e., $n/3$ [29]. To do so, the liveness guarantee is weakened. The idea is to modify an existing partially synchronous BFT protocol [1, 51] and make it "recoverable". Our AFD mechanism shares some similarity with this work in terms of detecting forks. The main difference is that we treat the underlying consensus mechanism (by a group of nodes) as a black-box and detect the forks via external groups.

3 SYSTEM MODEL AND BUILDING BLOCKS

We consider a system with n replicas $\{P_1, \dots, P_n\}$. The system tolerates f Byzantine faulty replicas and we assume $f < (1/3 - \epsilon)n$, where $\epsilon \in (0, 1/3)$. Non-faulty replicas are called correct replicas. We consider a static adversary model, where the adversary corrupts the replicas before the system is up and running. This is reasonable for a sharding-based approach, as pointed out by previous works [20]. We summarize some of our notations in Table 2.

Atomic broadcast (ABC). We consider atomic broadcast [13] in this work, which is only syntactically different from conventional

Table 2: Notations used in this work.

Notation	Description
λ	number of replicas in each shard
δ	probability of each shard being a faulty shard
$S_1, S_2, \dots$	shards in the system
$\mathcal{F}(S)$	the actual number of faulty replicas in shard S
P_i	replica i
C_i	the chain of blocks <i>a-delivered</i> by P_i
$\text{head}(C_i)$	the latest block in C_i
$\text{len}(C_i)$	the number of blocks of chain C_i
$C_i^{\uparrow k}$	the chain resulting from pruning k rightmost blocks from C_i
$\mathcal{WQ}(S)$	$\lfloor \frac{ S -1}{3} \rfloor + 1$, the size of weak quorum for shard S
$\mathcal{Q}(S)$	$\lceil \frac{ S + \lfloor \frac{ S -1}{3} \rfloor + 1}{2} \rceil$, the size of regular quorum for shard S
$\text{hash}()$	hash function
$C_i \not\preceq C_j$	C_i is not a prefix of C_j
$C_i \overset{\text{fork}}{\leftrightarrow} C_j$	forks, i.e., $C_i \not\preceq C_j$ and $C_j \not\preceq C_i$

BFT protocols. Informally, the main difference between ABC and BFT is that ABC does not explicitly consider the role of the "clients" who submit transactions to the system. Following the notion of blockchains, we define atomic broadcast as follows. Some replicas *a-broadcast* blocks and all replicas *a-deliver* some blocks. Each block m may contain a batch of transactions. An atomic broadcast protocol satisfies the following properties with probability $1 - \text{negl}(\kappa)$.

- **Safety:** If a correct replica *a-delivers* a block m before *a-delivering* m' , then no correct replica *a-delivers* a block m' without first *a-delivering* m .
- **Liveness:** If a correct replica *a-broadcasts* a block m , then all correct replicas eventually *a-deliver* m .

The API of ABC does not clearly specify the order of a transaction. But in practice, most protocols associate each transaction/block with a *sequence number*, a *height*, or an *epoch number*. We use epoch numbers in this work to specify the order of a block.

It is widely known that the security properties of ABC hold for a system with n replicas only if there are fewer than $f \leq \lfloor n/3 \rfloor$ faulty replicas in the partially synchronous network. In other cases, the properties above do not hold. Later in this work, we present our notion of multi-shard coordinating atomic broadcast (MC-ABC) that aims to handle the case where a shard has more than one-third faulty replicas.

Also note that the security properties we define above are slightly different from those in [13]. For example, our notion of ABC does not have the *agreement* property, i.e., if a correct replica *a-delivers* a block m , any correct replica eventually *a-delivers* m . We argue that this property is not a *must* in ABC and BFT, especially for partially synchronous protocols [16, 23, 52].

Sharding. Replicas are divided into different shards $\{S_1, S_2, \dots\}$. Each shard has a unique identifier. We assume that replicas are randomly divided into shards after the system is up and running. In a static adversary model, this means that the probability that each shard has more than two-thirds of correct replicas can be modeled as a binomial distribution. The identities of replicas in each shard are made publicly available.

Entry shard. We do not study how transactions are *allocated* to different shards and assume that the transactions are already allocated to the shards. We can use either the account model or the

UTXO model. Given each transaction tx , the shard tx is allocated for is called the *entry shard*. The shard is denoted as $\text{Entry}(tx)$. We may use m to denote a batch of transactions allocated to some shard and the entry shard is $\text{Entry}(m)$.

Correct shard vs. faulty shard; shard size. Each shard may have more than one-third of faulty replicas. If so, the shard is called a *faulty shard*. Otherwise, the shard is called a *correct shard*.

Depending on the size of n and the shard size, the concrete probability that a shard is faulty can be determined. Since replicas are randomly allocated to each shard, we assume that the probability that each shard is a faulty shard is δ . If the shard size is large enough, δ is a negligible function (see Lemma D.1 of Appendix D). Here, λ is a security parameter independent of the system size n . To differentiate λ from the security parameter κ (which often denotes the length of the hash function or digital signatures), we intentionally differentiate λ from κ in this work. One may consider the security parameter of the system as $\max(\lambda, \kappa)$.

Chain of blocks. We borrow the notations of formalization of Nakamoto Consensus (NC) [26, 27] to model the order of blocks seen by correct replicas. The first block in the system is called the *genesis block*. Let C_j denote the chain of blocks *a-delivered* by some correct replica P_j . $C_j = \perp$ if there is no block in the chain. We may also say C_j is a chain *led* by block $\text{head}(C_j)$. If P_j proposes a new block m according to C_j , m becomes the new head of the chain and we say m *extends* C_j . Two blocks m and m' are called *conflicting blocks* if m is not an extension of the chain led by m' and m' is not an extension of the chain led by m .

Consider a chain C_j with length $k = \text{len}(C_j)$ and any non-negative $k' < k$, $C_j^{\lceil k' \rceil}$ denotes the chain resulting from *pruning* k rightmost blocks. Note that if $k' \geq k$, $C_j^{\lceil k' \rceil} = \perp$.

Given two chains C_i and C_j , we write $C_i \preceq C_j$ if C_i is a prefix of C_j . If $C_i \not\preceq C_j$ and $C_j \not\preceq C_i$, C_i and C_j are called *forks*, denoted as $C_i \xleftrightarrow{\text{fork}} C_j$.

Remark. In NC, the above formalization models the chains *observed* by replicas and correct replicas eventually *converge* to the same chain. There is no notion for *finalization* of blocks. In our work, we use the notations for achieving safety under a faulty shard. Therefore, the C_j notation denotes the *a-delivered* blocks by some correct replica P_j . If a shard has more than one-third of faulty replicas, forks may exist. We design a protocol that controls the number of forks to a constant number and allows some correct replicas to *abort* the protocol after detecting the forks.

Byzantine quorum and quorum certificates. A Byzantine quorum is a set of replicas. In the distributed system literature, given n replicas among which at most f can be Byzantine, a Byzantine quorum consists of $\lceil \frac{n+f+1}{2} \rceil$ replicas in the partially synchronous network [13]. If $n = 3f + 1$, a Byzantine quorum has $2f + 1$ replicas. Signatures from a Byzantine quorum are called a quorum certificate (QC) in many recent works [23, 46, 52]. A set of $f + 1$ replicas is called a *weak* Byzantine quorum and signatures from a weak Byzantine quorum are called a weak quorum certificate (wQC) [23]. A QC with $2f + 1$ signatures is also called a *regular* QC (rQC).

In this work, we divide replicas into different shards. We thus define some functions for Byzantine quorums and quorum certificates. Given a group of replicas S (e.g., a shard that consists of a subset of λ replicas), $Q(S)$ is the size of a (regular) Byzantine quorum of S and $WQ(S)$ is the size of a weak Byzantine quorum of S .

We use $\text{epoch}(qc)$ to denote the epoch number for the block corresponding to QC qc .

Common coins. We use a common coin primitive for sampling coordinating shards, a notion first introduced by Rabin [41]. The common coin abstraction can be instantiated in different ways. For example, by assuming a cryptographic trusted setup, we can use threshold signature [10] or threshold PRF [15]. In this case, for a system with n replicas, each replica sends a κ -bit string to all replicas and the total communication is κn^2 , where κ is the length of the security parameter.

We use the $\text{Coin}(x)$ notation for common coins where x is the input of the function. The $\text{Coin}(x)$ function returns the identity of a random shard S_i together with a proof σ . The proof σ is publicly verifiable such that any replica can verify whether S_i is the coordinating shard. We discuss our instantiation of the $\text{Coin}()$ function in Appendix C.

Cryptographic primitives. We use digital signatures in this work and we assume the unforgeability property. We use collision-resistant hash functions.

4 DEFINITION OF MC-ABC AND OVERVIEW OF OTTER

We take two steps to achieve the sweet spot of moderate shard size and overwhelming probability of correctness. First, we exploit the idea of sampling several coordinating shards to "facilitate" the agreement for an entry shard. The idea is to randomly sample a few shards to participate in the consensus protocol. In this way, the probability of correctness is uniform for *all* transactions, regardless of which entry shard the transactions are allocated to. Our construction translates to the *normal-case* operation of our Otter protocol. The design allows for a shard size of only over 400 to achieve both near-optimal resilience and an overwhelming probability of correctness, in contrast to over 1,200 in prior works.

Second, we want to lower the shard size further (to around 100) for the system to achieve decent performance. Indeed, most conventional ABC protocols achieve decent performance for a system with around 100 replicas. However, the probability that each shard has more than one-third of faulty replicas also increases when the shard size decreases. If a shard is faulty, safety cannot be guaranteed anymore. We thus model the safety of each shard as a problem of *fork detection* and design an abortable fork detection (AFD) primitive for this purpose. Our model allows the number of forks to be reduced to a constant number while allowing correct replicas to detect whether forks have existed. Eventually, active correct replicas will stay on the same *longest* chain with an overwhelming probability. Our construction of AFD translates to the rotating leader protocol of Otter, which also requires a coordinating shard to "help" with the fork detection.

In this section, we define MC-ABC and provide an overview of our Otter protocol.

4.1 Defining Multi-Shard Coordinating Atomic Broadcast (MC-ABC)

Coordinating shard. Given each transaction tx , several shards are sampled to participate in the consensus protocol. In our protocol, two shards are sampled during the normal-case protocol and one shard is sampled during epoch change (i.e., rotating leader). Given a shard $S_e = \text{Entry}(m)$ and an epoch number r , we use $S_{1,r}$ and $S_{2,r}$ to denote the coordinating shards. Each replica P_i can use $\text{Coord}(j, e, 1, r)$ and $\text{Coord}(j, e, 2, r)$ to verify whether replica P_j belongs to a coordinating shard. For example, P_j belongs to the first coordinating shard if $\text{Coord}(j, e, 1, r)$ returns true.

Active replica. Every correct replica may *abort* according to our notion. A replica that aborts will stop participating in the protocol and it becomes *inactive*. According to our specification, a correct replica only aborts after it detects forks. The replicas that have not aborted are called *active replicas*. Our notion of MC-ABC does not consider whether inactive replicas can become active again.

Defining MC-ABC. MC-ABC is defined as a primitive for a particular shard S_e . In this case, S_e is an entry shard responsible for certain transactions. We only care about the safety and liveness properties of the transactions allocated to S_e . We focus on *intra-shard* transactions (transactions that only need to be executed by one shard), and discuss cross-shard transactions in Appendix B.

MC-ABC is specified by *a-broadcast* and *a-deliver* events, same as those in ABC. For the security properties of MC-ABC, we consider replicas in S_e only, and the transactions allocated to S_e . Several shards other than S_e might be sampled as coordinating shards. We assume that there exists a sufficiently large pool of correct shards as candidates for coordinating shards. An MC-ABC protocol that handles intra-shard transactions satisfies the following properties with probability $1 - \text{negl}(\kappa)$.

- **Universal safety.** If S_e is a correct shard and a correct replica *a-delivers* a transaction tx before *a-delivering* tx' , then no correct replica *a-delivers* tx' without first *a-delivering* tx .
- **Intra-shard liveness.** If S_e is a correct shard and a correct replica in S_e *a-broadcasts* an intra-shard transaction tx , then all correct replicas in S_e eventually *a-deliver* tx .

The properties above assume S_e is a correct shard so correctness follows that of conventional ABC protocols.

For the case where the entry shard is faulty, we need to carefully define the security properties. In fact, the problem of faulty shard detection is a lot like failure detection. It is well known that no perfect Byzantine failure detector exists [13, 24, 32]. Thus, it is not surprising that one cannot design a perfect faulty shard detection. For example, if there are more than one-third of faulty replicas, the liveness of the system will never be achieved in a partially synchronous or an asynchronous network. Indeed, Byzantine replicas only have to remain silent, so no progress can be made. Our definitions of security thus only focus on safety. In particular, the security properties below hold with probability $1 - \text{negl}(\kappa)$.

- **Common prefix.** Let S_e be a faulty shard and let the number of correct replicas in S_e be $\Omega(|S_e|)$. If an active correct replica P_i maintains a chain C_i and an active correct replica P_j maintains a chain C_j , then $C_i^{[k]} \preceq C_j$ and $C_j^{[k]} \preceq C_i$ for a constant k .

- **Bounded forks.** Let S_e be a faulty shard and $Q(S_e) - \mathcal{F}(S_e) = O(|S_e|)$. Let I be a set of replicas such that for any $P_i, P_j \in I$, $C_i \xrightarrow{\text{fork}} C_j$. The size of I is bounded by a constant, i.e., $|I| = O(1)$.

Our properties about common prefix and bounded forks additionally assume that the number of correct replicas in S_e is $\Omega(|S_e|)$. We argue that this assumption is reasonable. Namely, replicas are randomly allocated to the shards. If each shard has $O(\lambda)$ size, the probability that a shard has at least one-third correct replicas is

bounded by $1 - \delta^{\frac{1}{3} - \frac{\epsilon^2}{2}}$, where δ is the probability of each shard being faulty. The probability $1 - \delta^{\frac{1}{3} - \frac{\epsilon^2}{2}}$ is significantly higher than $1 - \delta$ (see Lemma D.2 in Appendix D). As a result, we do not need a big shard size for the probability to be overwhelmingly large.

ABC vs. MC-ABC; Modeling finalization of blocks. We intentionally follow the notations of ABC to model MC-ABC. In particular, the unanimous safety and intra-shard liveness properties are the *sharding* version of ABC. Meanwhile, the common prefix and bounded forks properties are carefully defined to model the forks of correct replicas. Namely, it is possible that two correct replicas *a-deliver* conflicting blocks. However, with $1 - \text{negl}(\kappa)$ probability, at least one correct replica will *detect* the fork and then abort. Accordingly, consider an active correct replica P_i that maintains a chain C_i . After C_i is extended by a constant number of blocks, with an overwhelming probability, C_i is indeed the *longest chain*.

Later we show that, as C_i continuous to be extended (i.e., considering C_i' is the latest chain maintained by P_i , $C_i \preceq C_i'$), the probability that C_i is finalized grows as C_i' further grows and the probability becomes closer to 1.

4.2 Overview of Otter

Otter is epoch-based and each shard has its own unique epoch number. Shards do not have to share the same epoch number. For a particular shard S_e , the epoch number grows monotonically. In each epoch for a shard S_e , there is a unique leader, the identity of which is known by all replicas in S_e . Following the convention of partially synchronous BFT, there are two sub-protocols in Otter for each shard: a normal-case protocol and a rotating leader (view change) protocol. The two sub-protocols together make Otter a secure MC-ABC protocol.

The normal-case protocol (Sec. 5): Making the probability of achieving safety and liveness a uniform probability function for all transactions. The normal-case protocol aims to make the probability of achieving safety and liveness uniform and not dependent on the entry shard. Indeed, considering that certain transactions are allocated to some shard S_e , the probability of correctness violation is *fixed*. If S_e is faulty, none of the transactions can be safely delivered. The probability of safety violation for all transactions in the system is thus not the same. To make the probability of achieving safety and liveness uniform for all transactions/shards, we use several coordinating shards for the block in epoch r for each S_e . The coordinating shards are randomly sampled, and they are involved in the normal-case protocol. Our approach ensures that if x coordinating shards are sampled, the probability of achieving safety and liveness is $1 - \delta^x$. The probability holds for transactions allocated to any shards.

Additionally, doing so can balance the shard size and the probability of achieving safety and liveness. Informally, if x is larger, we can allow for a larger δ and have a smaller shard size! A trade-off here is that if x is larger, the latency of the protocol will become larger (so the throughput might be lower). In this work, we set x as two. Jumping ahead a little bit, as discussed in Sec. 7, by having a shard size of over 400, we can already simultaneously achieve near-optimal resilience (i.e., $f \in [\frac{1}{4}n, \frac{1}{3}n)$) and overwhelming probability of correctness (i.e., $1 - 10^{-10}$, roughly $1 - 2^{-30}$). In contrast, prior works assume a shard size of over 1,200 to achieve the same probability of correctness.

The main technical challenge we resolve is how to build an efficient protocol from sampling coordinating shards. As we later show in Sec. 5, while our protocol inherits some features from known protocols such as PBFT [16] and HotStuff [52], building a correct protocol and proving the correctness are quite challenging.

The rotating leader protocol (Sec. 6): Handling faulty shards via abortable fork detection (AFD). Although reducing the shard size from 1,200 to 400 is an improvement, most BFT protocols still suffer from low performance with over 400 replicas. We want to lower the shard size further.

We design a *rotating leader* protocol that naturally handles faulty shards. The goal is to handle the case that S_e is faulty and allow replicas to *detect* forks, i.e., whether some correct replica P_i *a-delivers* m and some other correct replica P_j *a-delivers* m' such that m is conflicting with m' . We propose the AFD primitive to achieve such a goal. The idea is that once a fork exists, at most one chain can be extended in each epoch. Eventually, with an overwhelming probability, some correct replicas will *abort* and active correct replicas eventually *converge* on the same *longest* chain.

To better illustrate our protocol, we first present an approach that achieves AFD under the existence of a trusted party. Assuming the existence of a trusted party allows us to focus on the core idea of fork detection. We then relax the assumption and show how Otter achieves AFD. Briefly speaking, the idea is to query another coordinating shard $S_{0,r}$ during the rotating leader protocol for each epoch r . The shard $S_{0,r}$ helps correct replicas in S_e detect whether a fork exists. If a fork exists, the leader in epoch r is correct, and $S_{0,r}$ is a correct shard, at most one chain will be extended. Eventually, some correct replicas will abort. The problem is thus close to the Nakamoto Consensus (NC), where correct replicas (called miners, to be specific) eventually converge on the longest chain. We model AFD similarly to NC for ease of understanding.

5 THE NORMAL-CASE PROTOCOL OF OTTER

We present the normal-case protocol for Otter in this section. We focus on intra-shard transactions in this section.

An attempt. As mentioned in Sec. 4.2, our idea is to sample two random coordinating shards to participate in the agreement on a block m for epoch r . We illustrate the naive approach in Algorithm 1. The workflow is similar to many partially synchronous BFT [11, 16, 22] besides that different shards execute different phases. For example, for readers who are familiar with partially synchronous BFT, such as PBFT [16], the protocol is similar to a four-phase version of PBFT. One may wonder why three phases are not sufficient. One note we would like to mention is that if the leader of S_e is correct and S_e

Algorithm 1 An attempt. Pseudocode for replica P_i . S_e is the entry shard.

```

1: initialization: let epoch  $r \leftarrow 1$ ,  $Q$  be the queue of pending transactions
2:  $\triangleright$  Propose phase, executed by  $S_e$ 
3: upon  $a\text{-deliver}(m)$ 
4:   if  $P_i$  is the leader in  $S_e$  then
5:     obtain a batch of transactions  $m$  from  $Q$ 
6:     send (PROPOSE,  $e, r, m$ ) to all replicas in  $S_e$ 
7:  $\triangleright$  Prepare phase, executed by  $S_e$ 
8: upon receiving (PROPOSE,  $e, r, m$ ) from the leader  $P_j$  for epoch  $r$ 
9:    $S_{1,r}, \sigma_1 \leftarrow \text{Coin}(e||r||1)$   $\{\text{//the first coordinating shard}\}$ 
10:  send a (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) message to all replicas in  $S_{1,r}$ 
11:  $\triangleright$  Commit phase, executed by  $S_{1,r}$ 
12: upon receiving (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) from  $P_j$ 
13:  verify that  $\text{Coord}(i, e, 1, r)$  returns true based on  $\sigma_1$ 
14:  wait for  $Q(S_e)$  matching (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) messages from  $S_e$ 
15:   $S_{2,r}, \sigma_2 \leftarrow \text{Coin}(e||r||2)$   $\{\text{//the second coordinating shard}\}$ 
16:  send a (COMMIT,  $e, r, \text{hash}(m), \sigma_1, \sigma_2$ ) to all replicas in  $S_{2,r}$ 
17:  $\triangleright$  Finalize phase, executed by  $S_{2,r}$ 
18: upon receiving a (COMMIT,  $e, r, \text{hash}(m), \sigma_1, \sigma_2$ ) message from  $P_j$ 
19:  let  $S_{1,r}$  be the shard  $P_j$  belongs to
20:  verify that  $\text{Coord}(j, e, 1, r)$  returns true based on  $\sigma_1$ 
21:  verify that  $\text{Coord}(i, e, 2, r)$  returns true based on  $\sigma_2$ 
22:  wait for  $Q(S_{1,r})$  matching (COMMIT,  $e, r, \text{hash}(m), \sigma_1$ ) messages from  $S_{1,r}$ 
23:  send a (FINALIZE,  $e, r, \text{hash}(m), \sigma_2$ ) to replicas in  $S_e$ 
24:  $\triangleright$  executed by  $S_e$ 
25: upon receiving (FINALIZE,  $e, r, \text{hash}(m), \sigma_2$ ) from  $P_j$ 
26:  let  $S_{2,r}$  be the shard  $P_j$  belongs to
27:  verify that  $\text{Coord}(j, e, 2, r)$  returns true based on  $\sigma_2$ 
28:  wait for  $Q(S_{2,r})$  matching (FINALIZE,  $e, r, \text{hash}(m), \sigma_2$ ) messages from  $S_{2,r}$ 
29:   $a\text{-deliver}(m)$   $\{\text{//synchronize with other replicas in } S_e \text{ if } m \text{ has not been received}\}$ 

```

and $S_{1,r}$ are both correct shards, three phases are sufficient for the protocol to achieve unanimous safety and intra-shard liveness. We present a four-phase protocol mainly because we want to balance the shard size and the probability of achieving safety and liveness. Namely, the probability of achieving safety and liveness depends on how many coordinating shards are sampled. We can have an even smaller shard size if we sample more coordinating shards. However, if we choose more coordinating shards, the latency of the normal-case protocol is also increased and the throughput is decreased. We therefore choose two in Otter.

The protocol in Algorithm 1 is not correct if any of S_e , $S_{1,r}$, and $S_{2,r}$ is a faulty shard. Both safety and liveness challenges arise. First, on the liveness side, progress cannot be made if some shard is faulty. For example, if the leader in epoch r is correct, S_e is correct, but $S_{1,r}$ is incorrect, epoch r cannot be completed. In this case, we do not want to *waste* the messages exchanged among replicas in S_e . We introduce a *liveness QC* to make sure that m is still *useful* in epochs greater than r . Second, the safety issue is even trickier. If S_e is faulty, safety cannot be achieved. To deal with this, we borrow the concept of *lockedQC* from HotStuff [52] (which is similar to the *prepare certificate* of PBFT [16]), and use it for abortable fork detection during rotating leader protocol (to be described in Sec. 6).

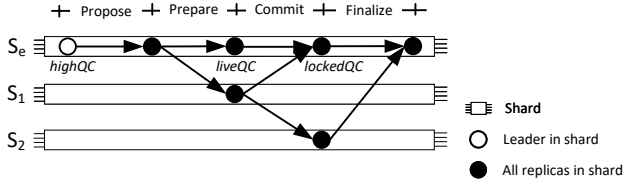


Figure 2: The normal-case protocol of Otter.

The normal-case protocol. We present the normal-case protocol of Otter in Algorithm 2 and illustrate it in Figure 2. We highlight the changes made on top of our attempt in gray. There are two major changes. First, replicas in S_e act as *learners* in the commit and finalize phases. Namely, replicas in $S_{1,r}$ and $S_{2,r}$ also send their messages to S_e . After receiving the messages, replicas in S_e also update their local state.

Second, we introduce three types of QCs: *lockedQC*, *liveQC*, and *highQC*. Only replicas in S_e maintain these QCs. The *liveQC* and *lockedQC* are updated in the finalize and commit phases, respectively. The purpose of *lockedQC* is similar to conventional BFT protocols [52]. It ensures that if S_e and $S_{1,r}$ are correct, correct replicas in S_e will always *a-deliver* the same messages. Additionally, *liveQC* is the liveness QC mentioned above to handle the case where either $S_{1,r}$ or $S_{2,r}$ is faulty. Indeed, if either $S_{1,r}$ or $S_{2,r}$ is faulty, no progress can be made in epoch r . In Otter, upon receiving $Q(S_e)$ matching (PREPARE) messages, each replica in S_e updates its *liveQC*. The *liveQC* is exchanged during the rotating leader protocol, and the leader in each epoch obtains the highest one (i.e., *highQC*). In this way, those proposals from correct leaders in S_e are not *wasted*.

Complexity. The message complexity of the normal-case protocol is $O(\lambda^2)$. The communication complexity for S_e and each coordinating shard is $O(L\lambda + \kappa\lambda^2)$ and $O(\kappa\lambda^2)$, respectively. We show the details in Appendix A.2.

6 THE ROTATING LEADER PROTOCOL OF OTTER

As mentioned in Sec. 4.2, we study the detection of faulty shards via the rotating leader protocol. We model the problem as fork detection, and we propose the notion of abortable fork detection (AFD). The abstraction of AFD resembles the Nakamoto consensus (NC). In NC, each replica (i.e., a miner) maintains its own chain, and forks might occur. Eventually, correct replicas will detect the longer chain, and correct replicas eventually converge to the longest chain. As the chain that extends a block m grows, the probability that a fork of m belongs to the longest chain of other correct replicas decreases. Although NC does not clearly have a notion of *finalization*, it is highly possible that m is *finalized* (i.e., *a-delivered*) and the chain led by m is indeed the longest chain.

In AFD, we model the blocks each replica *a-delivers* as a chain. If forks exist in shard S_e , correct replicas will eventually detect them with the help of other coordinating shards. Replicas that detect the fork immediately abort and stop participating in the protocol. The probability that no correct replica detects the forks decreases as the chain grows. With reasonable assumptions, the number of forks is a constant. In this way, even if some shard is faulty, replicas can

Algorithm 2 The normal-case operation of Otter for intra-shard transactions. Pseudocode for replica P_i and S_e is the entry shard.

```

1: initialization: let epoch  $r \leftarrow 1$ ,  $Q$  be the queue of pending transactions,
    $lr \leftarrow 1$ ,  $lockedQC \leftarrow \perp$ ,  $highQC \leftarrow \perp$ ,  $liveQC \leftarrow \perp$ 
2:  $\triangleright$  Propose phase, executed by  $S_e$ 
3: upon  $a$ -broadcast( $m$ )
4:   if  $P_i$  is the leader for epoch  $r$  in  $S_e$  then
5:     obtain a batch of transactions  $m$  from  $Q$ 
6:     send (PROPOSE,  $e, r, m, highQC$ ) to all replicas in  $S_e$ 
7:  $\triangleright$  Prepare phase, executed by  $S_e$ 
8: upon receiving (PROPOSE,  $e, r, m, highQC$ ) from the leader  $P_j$ 
   s.t.  $m$  is not conflicting with  $lockedQC$  and  $highQC$  is the highest
    $liveQC$  in the  $M$  field of the (NEW-EPOCH) message (see Algorithm 5)
9:    $S_{1,r}, \sigma_1 \leftarrow \text{Coin}(e||r||1)$ 
10:  send (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) to  $S_{1,r}$  and  $S_e$ 
11:  $\triangleright$  Commit phase, executed by  $S_{1,r}$  and  $S_e$ 
12: upon receiving (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) from  $P_j$  and  $P_j \in S_e$ 
13:  verify that  $\text{Coord}(i, e, 1, r)$  returns true based on  $\sigma_1$  or  $P_i \in S_e$ 
14:  wait for  $Q(S_e)$  matching (PREPARE,  $e, r, \text{hash}(m), \sigma_1$ ) messages
   from  $S_e$  (denoted as  $pQC$ )
15:  if  $\text{Coord}(i, e, 1, r)$  returns true then
16:     $S_{2,r}, \sigma_2 \leftarrow \text{Coin}(e||r||2)$ 
17:    send (COMMIT,  $e, r, \text{hash}(m), \sigma_1, \sigma_2$ ) to  $S_{2,r}$  and  $S_e$ 
18:  else  $\{\forall P_i \in S_e\}$ 
19:    send (COMMIT,  $e, r, \text{hash}(m)$ ) to  $S_e$ 
20:    set  $liveQC$  as  $pQC$ 
21:  $\triangleright$  Finalize phase, executed by  $S_{2,r}$  and  $S_e$ 
22: upon receiving (COMMIT,  $e, r, \text{hash}(m), \sigma_1, \sigma_2$ ) from  $P_j$  and
    $\text{Coord}(i, e, 1, r)$  returns true based on  $\sigma_1$  or  $P_j \in S_e$ 
23:  verify that  $\text{Coord}(i, e, 2, r)$  returns true based on  $\sigma_2$  or  $P_i \in S_e$ 
24:  wait for  $Q(S_{1,r})$  (COMMIT,  $e, r, \text{hash}(m), *, *$ ) messages are re-
   ceived from  $S_{1,r}$  (denoted as  $cQC$ ); if  $P_i \in S_e$ , additionally wait for
    $Q(S_e)$  messages from  $S_e$ 
25:  set  $lockedQC$  as  $cQC$ ,  $\sigma_1$ 
26:  send (FINALIZE,  $e, r, \text{hash}(m), \sigma_2, lockedQC$ ) to  $S_e$ 
27:  $\triangleright$  Executed by  $S_e$ 
28: upon receiving (FINALIZE,  $e, r, \text{hash}(m), \sigma_2, qc$ ) from  $P_j$  and  $P_j \in S_e$ 
   or  $\text{Coord}(i, e, 2, r)$  returns true based on  $\sigma_2$ 
29:  wait for  $Q(S_{2,r})$  (FINALIZE,  $e, r, \text{hash}(m), \sigma_2, qc$ ) messages from
    $S_{2,r}$  and  $Q(S_e)$  messages from  $S_e$ 
30:   $a$ -deliver( $m$ ) and  $m$ 's ancestors
31:  enter epoch  $lr + 1$ 

```

still achieve the safety of some block m after the chain that extends m grows for a constant number of blocks.

6.1 Abortable Fork Detection (AFD)

To better describe our protocol in this subsection, we abstract away the state of correct replicas in the normal-case protocol. In this way, readers do not have to understand our normal-case protocol to understand our AFD mechanism.

In particular, every correct replica maintains the highest liveness QC (i.e., *liveQC*), the highest *lockedQC*, and the epoch number of

the latest *a-delivered* block (i.e., lr). Among the two QCs, *liveQC* is from $Q(S_e)$ replicas, and *lockedQC* is from $Q(S_{1,r})$ replicas.

When S_e is a correct shard, replicas naturally form the *weak support* relationships, as summarized below. In fact, many ABC/BFT protocols have such weak support relationships.

- **Observation 1:** If a correct replica P_i in S_e *a-delivers* a block m , then at least $\mathcal{W}Q(S_e)$ correct replicas set their *lockedQC* as a QC for some block m' such that $m' = m$ or m' extends m .
- **Observation 2:** If a correct replica P_i in S_e sets its *lockedQC* for some block m , then at least $\mathcal{W}Q(S_e)$ correct replicas set their *liveQC* to a QC for some block m' such that $m' = m$ or m' extends m .

Based on the above observations, we present constructions that satisfy AFD. Our ultimate goal is to achieve the *common prefix* and *bounded forks* properties of MC-ABC. To better show our protocol, we first introduce two constructions based on the fact that a trusted third party exists. Then, we present a construction without the trusted party, i.e., our rotating leader protocol.

For readers who are familiar with ABC, one can directly read Sec. 6.3 to learn our concrete construction.

6.2 Attempts under the Existence of a Trusted Third Party

We provide two toy constructions assuming the existence of a trusted third party (again, to motivate our final design that does not rely on a trusted third party). The trusted party serves as an oracle O that receives a query and returns a correct result. We model the property of fork detection as follows. All replicas are from the same shard S_e . We consider an O_{all} oracle that always sends a response to *all* replicas and an O_{single} oracle that sends a response to one replica at a time. We consider the following properties for AFD.

- **Instant fork detection.** Let O_{all} send its response to *all* replicas. In some epoch r , a correct replica P_i maintains a chain C_i , a correct replica P_j maintains a chain C_j and $C_i \xleftrightarrow{\text{fork}} C_j$. If $Q(S_e)$ replicas query the O_{all} , at least one replica will abort.
- **Eventual fork detection.** Let O_{single} send its response to one replica at a time and provide proof of its response. In some epoch r , if a correct replica P_i maintains a chain C_i , a correct replica P_j maintains a chain C_j and $C_i \xleftrightarrow{\text{fork}} C_j$, at most one of the replicas will extend its chain in each epoch. Eventually, one of the replicas will abort.

▷ $\Pi_{\text{trusted}}^{\text{instant}}$. We first describe a protocol $\Pi_{\text{trusted}}^{\text{instant}}$ that satisfies instant fork detection in Algorithm 3. Here, the oracle is denoted as O_{all} .

We discuss why $\Pi_{\text{trusted}}^{\text{instant}}$ achieves the instant fork detection property. The O_{all} oracle receives (QUERY) messages from S_e and then forwards these messages to all replicas in S_e . As O_{all} is a trusted party, both P_i and P_j receive the same (STATE) message. Let $\text{head}(C_i)$ and $\text{head}(C_j)$ be m_i (proposed in epoch r_i) and m_j (proposed in epoch r_j), respectively. Consider the case where S_e is a correct shard. If P_i *a-delivers* m_i , observation 1 and 2 imply that the block m' for *liveQC* of $\mathcal{W}Q(S_e)$ replicas is either m_i or some block m' that extends m_i . As the (STATE) message consists of at least $Q(S_e)$ messages, at least one must be the *lockedQC* for m' . Accordingly, there are three cases (as illustrated in Figure 3a):

Algorithm 3 $\Pi_{\text{trusted}}^{\text{instant}}$. The fork detection protocol under the existence of a trusted third party with the instant fork detection property. Pseudocode for replica P_i in shard S_e .

```

1: ▷ Replica in  $S_e$ 
2: state of the replica: liveQC, lockedQC, current epoch  $lr$ 
3: upon entering epoch  $r = lr + 1$  or timeout in epoch  $lr$ 
4:   send (QUERY,  $e, r$ , liveQC) to  $O_{\text{all}}$ 
5:   wait until receiving (STATE,  $r$ ,  $M$ ) from  $O_{\text{all}}$ 
6:   let  $qc$  be the highest QC in the liveQC field of (QUERY) messages in  $M$ 
7:   if  $\text{epoch}(qc) < \text{epoch}(\text{lockedQC})$  or there exists at least one liveQC that is conflicting with lockedQC then
8:     abort
9:   else
10:    set  $lr \leftarrow lr + 1$  and resume normal-case operation
11: ▷ The  $O_{\text{all}}$  oracle
12: upon receiving  $Q(S_e)$  (QUERY) messages from  $S_e$ 
13:   let the  $Q(S_e)$  messages be  $M$ 
14:   send (STATE,  $r$ ,  $M$ ) to all replicas in  $S_e$ 

```

- (1) There exists a *liveQC* in the (QUERY) messages in M for m' (either $m' = m_i$ or m' is an extension of m_i);
- (2) There does not exist such a *liveQC*, and the epoch number of the highest QC is lower than $\text{epoch}(\text{head}(C_i))$;
- (3) There exists a *liveQC* in the (QUERY) messages in M for m'' and m'' is conflicting with m_i .

In the first case, as the *liveQC* for m' is conflicting with that for m_j , P_j will abort. In the second and the third cases, P_i will abort, or both P_i and P_j will abort.

Examples. We show three examples to explain the three cases above. In all three cases, let r_i be the epoch number of $\text{head}(C_i)$ and r_j be the epoch number of $\text{head}(C_j)$.

Figure 3b illustrates case (1). In particular, *liveQC* exists for m' that extends m_i . In this case, P_i will not abort. However, as P_j also receives the same M according to the assumption of O_{all} , m' is conflicting with $m_j = \text{head}(C_j)$. In this case, P_j will abort.

Figure 3c illustrates case (2). The epoch number of the highest epoch of *liveQC* is r' (for block m') and $r' < r_i$ and $r' < r_j$. In this case, both P_i and P_j will abort. To see why this is *needed* for AFD, consider that in a new epoch $r'' > \max(r_i, r_j)$. Even if the leader is correct, it will propose a new block that extends m' , and the chain led by m' conflicts with both C_i and C_j . Namely, this case violates observation 2.

Figure 3d illustrates case (3). Consider the case that the block for the highest *liveQC* is m' and m' is conflicting with both C_i and C_j . In this case, both P_i and P_j will abort.

▷ $\Pi_{\text{trusted}}^{\text{eventual}}$. We slightly revise $\Pi_{\text{trusted}}^{\text{instant}}$ and provide a construction that achieves the eventual fork detection property. We show the pseudocode in Algorithm 4. The main difference here is that instead of directly sending the (STATE) message to all replicas in S_e , the O_{single} oracle only sends the (STATE, r , M , π) message to the leader in epoch r , where π is a proof for M (e.g., a digital signature from O_{single}). In this case, the leader in epoch r may choose to *hide* the message and send the (NEW-EPOCH) message to some correct replicas, so the protocol needs to be adapted accordingly.

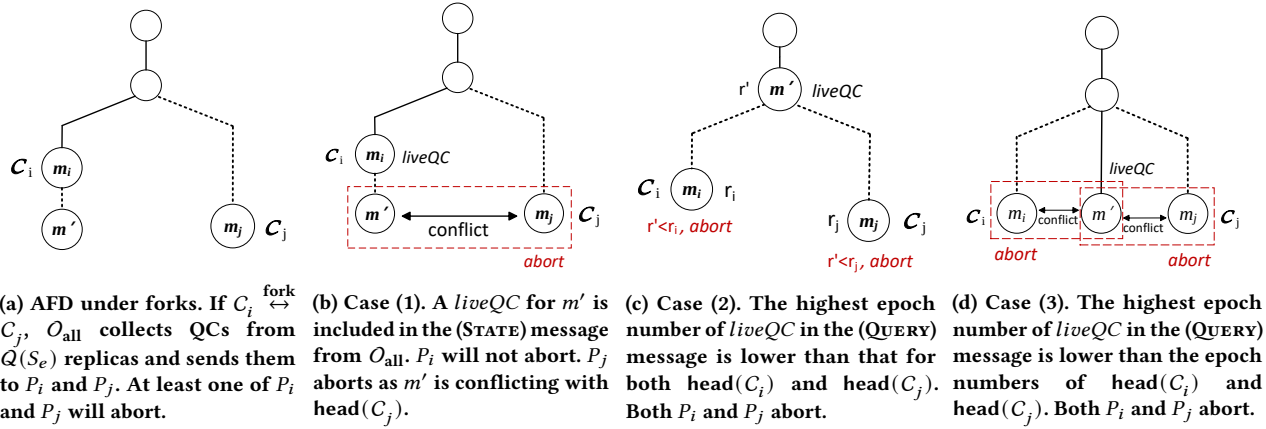


Figure 3: Examples of $\Pi_{\text{trusted}}^{\text{instant}}$.

Algorithm 4 $\Pi_{\text{trusted}}^{\text{eventual}}$: The fork detection protocol under the existence of a trusted third party with the eventual fork detection property. The differences on top of $\Pi_{\text{trusted}}^{\text{instant}}$ are highlighted in gray. Pseudocode for replica P_i in shard S_e .

```

1: ▷ Replica in  $S_e$ 
2: state of the replica:  $\text{liveQC}$ ,  $\text{lockedQC}$ , current epoch  $lr$ 
3: upon entering epoch  $r = lr + 1$  or timeout in epoch  $lr$ 
4:   send (QUERY,  $e, r, \text{liveQC}$ ) to  $O_{\text{single}}$ 
5:   if  $P_i$  is the leader in epoch  $r$  then
6:     wait until receiving (STATE,  $r, M, \pi$ ) from  $O_{\text{single}}$ 
7:     send (NEW-EPOCH,  $r, M, \pi$ ) to all replicas
8:   upon receiving (NEW-EPOCH,  $M, \pi$ ) from the leader for epoch  $r$  such
   that  $\pi$  is a valid proof for  $\text{hash}(M)$  from  $S_{0,r}$ 
9:     let  $qc$  be the highest QC in the  $\text{liveQC}$  field of (QUERY) messages
   in  $M$ 
10:    if  $\text{epoch}(qc) < \text{epoch}(\text{lockedQC})$  or there exists at least one
     $\text{liveQC}$  that is conflicting with  $\text{lockedQC}$  then
11:      abort
12:    else
13:      set  $lr \leftarrow lr + 1$  and resume normal-case operation
14: ▷ The  $O_{\text{single}}$  oracle
15: upon receiving  $Q(S_e)$  (QUERY) messages from  $S_e$ 
16:   let the  $Q(S_e)$  messages be  $M$ 
17:   send (STATE,  $r, M, \pi$ ) to leader of epoch  $r$  in  $S_e$ , where  $\pi$  is a proof
   for  $\text{hash}(M)$ 

```

We now discuss why $\Pi_{\text{trusted}}^{\text{eventual}}$ achieves the eventual fork detection property. First, the oracle O_{single} will only send one (STATE) message for each epoch r to the leader P_ℓ of epoch r . After the leader P_ℓ receives the (STATE, r, M, π) message, it may choose to send a (NEW-EPOCH, r, M, π) message to all replicas, where π is a proof for M . Therefore, if a correct replica P_i receives (NEW-EPOCH, r, M, π) and another correct replica P_j receives (NEW-EPOCH, r, M', π'), $M = M'$. Second, following the discussion of $\Pi_{\text{trusted}}^{\text{instant}}$, it is not too difficult to see that P_i and P_j will not both extend their chains in the same epoch r . Namely, if P_i extends its chain C_i in epoch r , the proposal

m'_i by the leader P_ℓ must be consistent with its lockedQC (i.e., a QC for $m_i = \text{head}(C_i)$). In this case, P_j will never consider the (NEW-EPOCH) message by P_ℓ valid. Accordingly, there are two cases.

- If P_j receives such a (NEW-EPOCH) message from P_ℓ , it directly aborts so eventual fork detection holds.
- If P_j does not receive the (NEW-EPOCH) message from P_ℓ , it still stays active. In another epoch (e.g., epoch $r + 1$), P_j may still extend its chain. If so, P_i will not extend its chain in epoch $r + 1$.

Eventually, some epoch r^* exists such that the leader P_k is correct. In this case, P_k will send a (NEW-EPOCH) message to all replicas in S_e . Then, at least one of P_i and P_j will abort, following a similar argument as that in $\Pi_{\text{trusted}}^{\text{instant}}$.

6.3 Rotating Leader Protocol in Otter: AFD without Trusted Third Party

We are now ready to present our rotating leader protocol, which can be viewed as an AFD protocol without a trusted third party. Informally, our rotating leader protocol uses coordinating shards to instantiate the O_{single} oracle. We now define the security property.

- **Extended eventual fork detection.** Let the number of correct replicas in S_e be $\Omega(|S_e|)$. In some epoch r , if a correct replica P_i maintains a chain C_i and a correct replica P_j maintains a chain C_j and $C_i \xrightarrow{\text{fork}} C_j$, then at most one of P_i and P_j will extend its chain in each epoch r with probability $1 - \text{negl}(\kappa)$. Eventually, one of the replicas will abort.

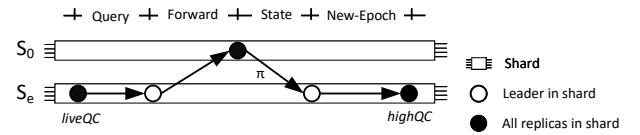


Figure 4: The rotating leader protocol of Otter.

We illustrate the workflow in Figure 4 and show the pseudocode in Algorithm 5. The idea is very straightforward once we have the construction $\Pi_{\text{trusted}}^{\text{eventual}}$. If we randomly sample a coordinating shard (denoted as $S_{0,r}$) during epoch changes, with a probability of $1 - \text{negl}(\kappa)$, $S_{0,r}$ is a correct shard. We can then treat $S_{0,r}$ as the

Algorithm 5 The rotating leader sub-protocol of Otter. The differences on top of $\Pi_{\text{trusted}}^{\text{eventual}}$ are highlighted in gray. Pseudocode for replica P_i in shard S_e .

```

1: ▷ Replica in  $S_e$ 
2: state of the replica: liveQC, lockedQC, current epoch  $lr$ 
3: upon entering epoch  $r = lr + 1$  or timeout in epoch  $lr$ 
4:   send (QUERY,  $e, r$ , liveQC) to leader of epoch  $r$  in  $S_e$ 
5:    $S_{0,r}, \sigma_3 \leftarrow \text{Coin}(e || r || 0)$ 
6: upon receiving  $Q(S_e)$  (QUERY) messages and  $P_i$  is the leader in
   epoch  $r$ 
7:   let  $Q(S_e)$  (QUERY) messages be  $M$  and  $h = \text{hash}(M)$ 
8:   send (FORWARD,  $e, r, h, \sigma_3$ ) to all replicas in  $S_{0,r}$ 
9: upon receiving  $Q(S_{0,r})$  matching (STATE) messages for hash( $M$ )
   from  $S_{0,r}$ 
10:  let  $\pi$  be the set of  $Q(S_{0,r})$  signatures in (STATE) messages
11:  send (NEW-EPOCH,  $r, M, \pi$ ) to all replicas
12: upon receiving (NEW-EPOCH,  $M, \pi$ ) from the leader for epoch  $r$  such
   that  $\pi$  is a valid proof for hash( $M$ )
13:  let  $qc$  be the highest QC in the liveQC field of (QUERY) messages
   in  $M$ 
14:  if  $\text{epoch}(qc) < \text{epoch}(\text{lockedQC})$  or there exists at least one
   liveQC that is conflicting with lockedQC then
15:    abort
16:  else
17:    set  $lr \leftarrow lr + 1$  and resume normal-case operation
18: ▷ Replica in  $S_{0,r}$ 
19: upon receiving a (FORWARD,  $e, r, h, \sigma_e$ ) from  $P_j$  in  $S_e$ 
20:  send (STATE,  $r, h, \pi_i$ ) to  $P_j$ , where  $\pi_i$  is a signature for  $h$ 

```

oracle O_{single} . To ensure that $S_{0,r}$ only sends the same message to all replicas S_e , we provide a protocol similar to that of consistent broadcast (CBC) [43] between the leader in epoch r and all replicas in $S_{0,r}$. In this case, a correct shard $S_{0,r}$ can be viewed as a trusted third party. We can then achieve eventual fork detection in each epoch with $1 - \text{negl}(\kappa)$ probability. Now, we are left to show how eventual fork detection leads to the common prefix and bounded forks properties.

Why does Otter achieve common prefix and bounded forks? As discussed above, in each epoch, with $1 - \text{negl}(\kappa)$ probability, Otter achieves the eventual fork detection property.

For the common prefix, the crux is to show that one of P_i and P_j eventually detects the fork and aborts within a constant number of epochs. Towards a contradiction, assume $C_i \xleftrightarrow{\text{fork}} C_j$. Let $C_i^{[k_1]} = C_j^{[k_2]}$, i.e., the fork starts to form in epoch $r+1$ such that $r = r_i - k_1 = r_j - k_2$. Let m be the block P_i and P_j *a-deliver* in epoch r . In some epoch $r' > r$, one of P_i and P_j will abort according to the eventual fork detection property. There are two conditions for one of them to abort in our construction: (1) the leader P_ℓ is correct, and it sends a (NEW-EPOCH) message to all replicas in S_e ; (2) the coordinating shard $S_{0,r'}$ is a correct shard. The probability of condition (1) is a constant as we assume that the number of correct replicas in S_e

is $\Omega(|S_e|)$, and each leader is randomly selected. Additionally, the probability of (2) is $1 - \text{negl}(\kappa)$. Therefore, it is not very difficult to see that eventually one of P_i and P_j will abort. The common prefix property thus holds.

The bounded forks property holds for two reasons. First, in each epoch, the number of forks can be formed is bounded by $\frac{S_e}{Q(S_e) - \mathcal{F}(S_e)}$, as an adversary needs $Q(S_e) - \mathcal{F}(S_e)$ matching votes from correct replicas for some correct replica to *a-deliver* a transaction. As we assume that $Q(S_e) - \mathcal{F}(S_e) = O(|S_e|)$, the number of forks in each epoch is then bounded by a constant. Second, according to the protocol, every correct replica only extends its local chain C_i and will not vote for a fork of C_i (due to the *lockedQC* design). Therefore, even if the forks are created in different epochs, the total number of forks in the system is still bounded by a constant.

We show the proof of Otter in Appendix D.

Complexity. The message complexity of the rotating leader protocol is $O(\lambda)$ as one replica interacts with all replicas in a shard at a time. The communication complexity for the shard S_e is $O(\kappa\lambda^3)$. This can be optimized to $O(\kappa\lambda^2 \log \lambda)$ by instantiating QC with aggregate signatures. The details can be found in Appendix A.2.

7 DISCUSSION

Analysis of the concrete probability. We show the relationships between ϵ (i.e., how much resilience needs to be sacrificed) and the shard size. To ensure that the probability of correctness is lower than $1 - 10^{-10}$, we show the concrete values in Table 3. It can be seen that our protocol makes it possible to select a moderate shard size to ensure that $f \in [\frac{1}{4}n, \frac{1}{3}n]$ (recall that if f is closer to $n/3$, the system achieves near-optimal resilience). Namely, we only need a shard size of over 400. In contrast, previous works need more than 1,200. Note that although some works claim that the safety and liveness thresholds can be tuned such that safety is not violated even if there are more than one-third of faulty replicas [20, 56], a shard needs at least two-thirds of correct replicas for the protocol to be both safe and live, a well known bound for any BFT protocols [13].

Since Otter has the AFD mechanism to detect the forks, we do not need such a high probability to have an even smaller shard size. For instance, to make the probability $1 - 10^{-4}$ (i.e., AFD is expected to be triggered once every 10,000 epochs), we only need a shard size of over 200. The shard size can be lowered to 100 to make the probability $1 - 10^{-3}$. Thus, with overwhelming probability, forks are eventually detected, if any. Consider that forks already exist in some epoch r . The probability that correct replicas still do not converge to the same chain in epoch $r + k$ is $1 - \delta^k$. As δ is a negligible function, $1 - \delta^k$ becomes closer to 1 when k grows.

Liveness of faulty shards. Our work focuses on safety in the presence of faulty shards and does not discuss liveness. In the introduction, we mentioned that one drawback of GearBox (and essentially all sharding-based protocols) is that we ultimately need a large shard size so that each shard is correct. One may wonder whether Otter indeed addresses the problem since GearBox does not suffer from safety issues. We want to emphasize that our protocol already supports a much smaller shard size to achieve the same probability of correctness compared to any existing works, as summarized in Table 3. Also note that unlike GearBox, in which

protocol	$n =$	1000	2000	3000	4000	5000
	$\lambda = ?n$					
previous works*	10%	0.233	0.183	0.145	0.128	0.118
	20%	0.183	0.128	0.108	0.098	0.083
	30%	0.145	0.108	0.088	0.076	0.069
	40%	0.128	0.098	0.076	0.069	0.059
Otter	10%	0.153	0.127	0.093	0.080	0.081
	20%	0.127	0.080	0.063	0.054	0.048
	30%	0.093	0.063	0.05	0.043	0.038
	40%	0.080	0.054	0.43	0.37	0.033

Table 3: The value of ϵ and shard size for the system to achieve safety and liveness with a probability of at least $1 - 10^{-10}$ (roughly $1 - 2^{-30}$). The system requires $f \in [0, \frac{1}{5}n]$, $f \in [\frac{1}{5}n, \frac{1}{4}n]$, and $f \in [\frac{1}{4}n, \frac{1}{3}n]$ for dark gray cells, gray cells, and white cells, respectively. *We consider that a shard must have at least two-thirds of correct replicas for the shard to be safe and live.

the actual number of failures in each shard needs to be *estimated*, our work does not require such an estimation.

To further ensure liveness, we may need to slightly relax the assumptions about the adversary and then integrate the notion of "recovery" [29, 35] with our approach. The notion of recovery aims to allow replicas to recover after the number of faulty replicas in a system exceeds the pre-specified threshold. In our context, such a mechanism allows correct replicas to become active again after they abort. We consider it an interesting future work.

Optimizing the computational complexity. In our protocol, the `Coin()` function needs to be queried three times for every epoch: twice for the normal-case protocol, and once during the epoch change. The computational cost might be high if we instantiate it using threshold signatures or threshold PRF [21, 38, 55]. One practical optimization is that we *fix* the leader and the two coordinating shards for several epochs and then sample them again. The proof of correctness and the probability analysis still hold as we assume a static adversary model.

8 IMPLEMENTATION AND EVALUATION

We implement Otter in Golang and will open source our codebase later. Our codebase involves around 10,000 LOC for the protocol and about 1,500 LOC for evaluation. In our implementation, we use gRPC as the communication library. We use HMAC to realize the authenticated channel and use SHA256 as the underlying hash function. We use threshold PRF [15] to instantiate the `Coin()` function. We show in Appendix C how the `Coin()` function is implemented using threshold PRF.

We evaluate the performance of our protocols on Amazon EC2 using up to 1,000 virtual machines (VMs). We use *m5.xlarge* instances for our evaluation. The *m5.xlarge* instance has four vCPUs and 16GB memory. We deploy our protocols in the WAN setting, where replicas are evenly distributed in four different regions: us-west-2 (Oregon, US), us-east-2 (Ohio, US), ap-southeast-1 (Singapore), and eu-west-1 (Ireland). By default, we deploy each shard in one region. We repeat each experiment 5 times and report the average performance.

Transactions are evenly spread across all shards. We use s to denote the total number of shards in the system. For our evaluation, we vary the network size n , the number of shards s , and the batch size (number of transactions in each proposal) b . In each setting with n replicas and s shards, the shard size is n/s . Given any fixed n and s , we vary b until the throughput does not increase (also known as peak throughput), but the latency keeps growing.

We use each EC2 instance to run two replicas. This is because we evaluated Otter using up to 1,000 replicas, but our account can not support such a large scale due to restrictions on the total number of vCPUs. To see the differences between running two replicas on each instance (denoted as *type1*) and running one replica on each instance (denoted as *type2*), we conduct experiments for $n = 500$ and $s = 4$ in both settings and report the results in Figure 5a. The performance of *type2* is consistently higher than that of *type1*, e.g., the peak throughput is 16.8% higher. As the performance is quite close, we use *type1* for all results reported in this section.

We summarize our evaluation results below.

- We compare the performance of Otter with GearBox [20], the state-of-the-art sharding-based consensus protocol. To have a fair comparison, we fix the probability of achieving safety and liveness to $1 - 10^{-9}$. Compared to GearBox, Otter requires only 55%-58% shard size to achieve the same probability of safety and liveness. Our evaluation result shows that the peak throughput of Otter is up to 4.38x that of GearBox.
- Otter is the first sharding protocol that is experimentally evaluated using 1,000 replicas in a WAN network. For instance, Arete [56] evaluates the performance using up to 500 replicas, GearBox [20] does not provide an implementation, and Reticulum [50] (a synchronous sharding protocol) is evaluated in LAN (and most experiments are simulations).
- Our evaluation of the latency breakdown shows that the overhead created by the coordinating shards is very low. In particular, the latency of the entry shard occupies 73%-82% of the overall latency and the propose phase (that carries bulk data) is the only bottleneck of the system.

Otter vs. GearBox. We compare the peak throughput of Otter with GearBox [20], using an open-source implementation³. In our evaluation, we first fix n and s for Otter, set up the desirable probability of correctness as $1 - 10^{-9}$, and then calculate the value of ϵ . Given the same probability and ϵ , we then determine the number of shards s' for GearBox. Similar to Table 3, s is much smaller than s' . We summarize the concrete numbers in Figure 5d. We then deploy both protocols and evaluate their peak throughput for $n = 500$, as shown Figure 5h. Our results show that when the value of ϵ equals 0.146, 0.197, and 0.249, the peak throughput of Otter is 1.71x, 2.7x, and 4.38x that of GearBox, respectively. This is mainly because Otter can use a much smaller shard size. Another reason is that GearBox uses all replicas in the system as the coordinating shard, but we randomly sample a shard as the coordinating shard.

Performance under different shard sizes. We first assess latency vs. throughput for Otter. We fix n and evaluate the performance by varying s . As shown in Figure 5a-5c, given the same n , the performance grows as s grows. This is expected, as there are fewer

³GearBox implementation by Arete [56]: <https://github.com/EtherCS/arete/tree/sota>

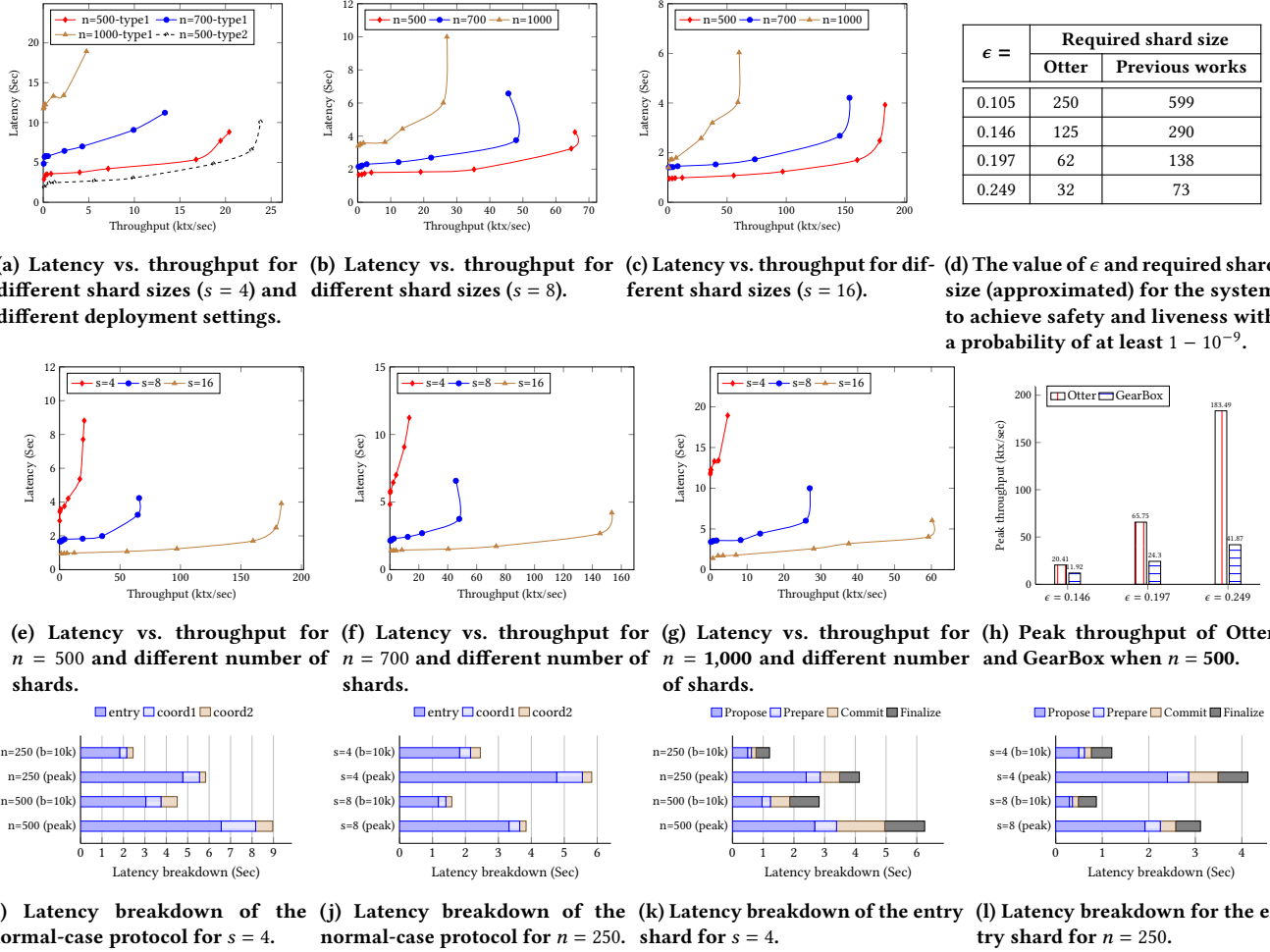


Figure 5: Evaluation results.

replicas in each shard, and a smaller number of nodes in each shard will lower the communication and computational costs. Our evaluation shows that Otter achieves the highest performance when $n = 500$ and $s = 16$, with a peak throughput of 183.49 ktx/sec. When the shard size is *fixed*, i.e., each shard has 62 replicas, the peak throughput for $n = 500$ and $s = 8$ setting is 65.75 ktx/sec, roughly the same as that for $n = 1,000$ and $s = 16$.

Performance under different numbers of shards. We also fix n to 500, 700, and 1,000 and assess the performance by varying the shard sizes. We report the performance in Figure 5e-5g. Note that Figure 5e-5g use the same data points as those in Figure 5a-5c. We intentionally do so as it is meaningful to report the performance both under different shard sizes and different numbers of shards. For each n/s , the performance degrades as n grows. This is expected as there are more replicas in each shard. In the setting with $n = 1,000$ and $s = 4$, the system achieves a peak throughput of 4.75 ktx/sec. For the setting with $n = 1,000$ and $s = 16$, our system achieves a peak throughput of 60.19 ktx/sec.

Latency breakdown. We report the latency breakdown of the normal-case protocol and also for different phases at the entry

shard. As shown in Figure 5i-5j, the latency of the entry shard is the bottleneck of the protocol. This is mainly because the propose phase run by the entry shard carries bulk data. Additionally, as shown in Figure 5k-5l, the entry shard participates in all four phases but the propose phase is still the bottleneck.

To further reduce the overhead caused by the propose phase, we can use the idea of de-coupling bulk data transmission from consensus, a useful paradigm that can improve the performance of the system [19, 23, 25].

9 CONCLUSION

We present Otter, a practical Byzantine fault-tolerant sharding-based atomic broadcast protocol. Otter is built from a new primitive called multi-shard coordinating atomic broadcast (MC-ABC). Our concrete construction achieves a nice balance between the shard size and the correctness of the system. To our knowledge, Otter is the first sharding-based system deployed with over 1,000 replicas in WAN while still achieving decent performance.

REFERENCES

- [1] 2022. Tendermint Core. <https://github.com/tendermint/tendermint>. (2022).
- [2] Ittai Abraham, TH Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. 2019. Communication complexity of byzantine agreement, revisited. In *PODC*. 317–326.
- [3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. 2017. Solida: A blockchain protocol based on reconfigurable Byzantine consensus. In *OPODIS*.
- [4] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2021. Good-Case Latency of Byzantine Broadcast: A Complete Categorization. In *PODC*.
- [5] Mustafa Al-Bassam, Alberto Sonnino, Shehar Bano, Dave Hrycyszyn, and George Danezis. 2018. Chainspace: A sharded smart contracts platform. In *NDSS*.
- [6] Yair Amir, Claudiu Danilov, Jonathan Kirsch, John Lane, Danny Dolev, Cristina Nita-Rotaru, Josh Olsen, and David Zage. 2006. Scaling Byzantine Fault-Tolerant Replication to Wide Area Networks. In *DSN*. IEEE, 105–114.
- [7] Vivek Bagaria, Sreeram Kannan, David Tse, Giulia Fanti, and Pramod Viswanath. 2019. Prism: Deconstructing the blockchain to approach physical limits. In *CCS*. 585–602.
- [8] Erica Blum, Elette Boyle, Ran Cohen, and Chen-Da Liu-Zhang. 2023. Communication Lower Bounds for Cryptographic Broadcast Protocols. In *DISC*.
- [9] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. 2020. Asynchronous byzantine agreement with subquadratic communication. In *TCC*. Springer, 353–380.
- [10] Alexandra Boldyreva. 2003. Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme. In *PKC*.
- [11] Ethan Buchman. 2017. Tendermint: byzantine fault tolerance in the age of blockchains. (2017).
- [12] Vitalik Buterin, Diego Hernandez, Thor Kampefner, Khien Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang, and Yan X Zhang. 2020. Combining GHOST and casper. *arXiv preprint arXiv:2003.03052* (2020).
- [13] Christian Cachin, Rachid Guerraoui, and Luis Rodrigues. 2011. *Introduction to reliable and secure distributed programming*. Springer Science & Business Media.
- [14] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*. Springer, 524–541.
- [15] Christian Cachin, Klaus Kursawe, and Victor Shoup. 2005. Random oracles in Constantinople: Practical asynchronous Byzantine agreement using cryptography. *Journal of Cryptology* 18, 3 (2005), 219–246.
- [16] Miguel Castro and Barbara Liskov. 2002. Practical Byzantine fault tolerance and proactive recovery. *TOCS* 20, 4 (2002), 398–461.
- [17] T-H Hubert Chan, Rafael Pass, and Elaine Shi. 2020. Sublinear-round byzantine agreement under corrupt majority. In *PKC*. Springer, 246–265.
- [18] Jing Chen and Silvio Micali. 2019. Algorand: A secure and efficient distributed ledger. *Theoretical Computer Science* 777 (2019), 155–183.
- [19] George Danezis, Eleftherios Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *Eurosys*. 34–50.
- [20] Bernardo David, Bernardo Magri, Christian Matt, Jesper Buus Nielsen, and Daniel Tschudi. 2022. Gearbox: Optimal-size shard committees by leveraging the safety-liveness dichotomy. In *CCS*. 683–696.
- [21] Sisi Duan, Xin Wang, and Haibin Zhang. 2023. Practical Signature-Free Asynchronous Common Subset in Constant Time. In *CCS*.
- [22] Sisi Duan and Haibin Zhang. 2022. Foundations of Dynamic BFT. In *IEEE Symposium on Security and Privacy (SP)*. 1317–1334.
- [23] Sisi Duan, Haibin Zhang, Xiao Sui, Baohan Huang, Changchun Mu, Gang Di, and Xiaoyun Wang. 2024. Dashing and Star: Byzantine Fault Tolerance from Weak Certificates. In *Eurosys*.
- [24] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. 1985. Impossibility of distributed consensus with one faulty process. *JACM* 32, 2 (1985), 374–382.
- [25] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2022. Dumbo-NG: Fast Asynchronous BFT Consensus with Throughput-Oblivious Latency. In *CCS*.
- [26] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. 2017. The bitcoin backbone protocol with chains of variable difficulty. In *CRYPTO*.
- [27] Juan A Garay, Aggelos Kiayias, and Nikos Leonardos. 2015. The bitcoin backbone protocol: Analysis and applications. *J. ACM* (2015).
- [28] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. 2017. Algorand: Scaling byzantine agreements for cryptocurrencies. In *SOSP*. ACM, 51–68.
- [29] Tiantian Gong, Gustavo Franco Camilo, Kartik Nayak, Andrew Lewis-Pye, and Aniket Kate. 2025. Recover from Excessive Faults in Partially-Synchronous BFT SMR. *Cryptology ePrint Archive* (2025).
- [30] Zicong Hong, Song Guo, Peng Li, and Wuhui Chen. 2021. Pyramid: A Layered Sharding Blockchain System. In *INFOCOM*. 1–10. <https://doi.org/10.1109/INFOCOM42981.2021.9488747>
- [31] Huawei Huang, Xiaowen Peng, Jianzhou Zhan, Shenyang Zhang, Yue Lin, Zibin Zheng, and Song Guo. 2022. Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding. In *INFOCOM*. IEEE, 1968–1977.
- [32] Kim Potter Kihlstrom, Louise E Moser, and P Michael Melliar-Smith. 2003. Byzantine fault detectors for solving consensus. *Comput. J.* 46, 1 (2003), 16–35.
- [33] Valerie King and Jared Saia. 2011. Breaking the $O(n^2)$ bit barrier: scalable byzantine agreement with an adaptive adversary. *JACM* 58, 4 (2011), 18.
- [34] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, and Bryan Ford. 2018. OmniLedger: A Secure, Scale-Out, Decentralized Ledger. In *S&P*.
- [35] Jacob D Leshno, Elaine Shi, and Rafael Pass. 2024. On the viability of open-source financial rails: Economic security of permissionless consensus. *arXiv preprint arXiv:2409.08951* (2024).
- [36] Mingzhe Li, You Lin, Jin Zhang, and Wei Wang. 2023. CoChain: High concurrency blockchain sharding via consensus on consensus. In *INFOCOM*. IEEE, 1–10.
- [37] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. 2016. A secure sharding protocol for open blockchains. In *CCS*. ACM, 17–30.
- [38] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The honey badger of BFT protocols. In *CCS*. 31–42.
- [39] Atsuki Momose and Ling Ren. 2021. Multi-threshold byzantine fault tolerance. In *CCS*. 1686–1699.
- [40] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).
- [41] Michael O Rabin. 1983. Randomized byzantine generals. In *FOCS*. IEEE, 403–409.
- [42] Sajad Rahnama, Suyash Gupta, Rohan Sogani, Dhruv Krishnan, and Mohammad Sadoghi. 2022. RingBFT: Resilient Consensus over Sharded Ring Topology. In *EDBT*. 2:298–2:311.
- [43] Michael K Reiter. 1994. Secure agreement protocols: Reliable and atomic group multicast in Rampart. In *CCS*. 68–80.
- [44] Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. 2021. BFT protocol forensics. In *CCS*. 1722–1743.
- [45] Srivatsan Sridhar, Dionysis Zindros, and David Tse. 2023. Better Safe than Sorry: Recovering after Adversarial Majority. *arXiv preprint arXiv:2310.06338* (2023).
- [46] Xiao Sui, Sisi Duan, and Haibin Zhang. 2022. Marlin: Two-Phase BFT with Linearity. *DSN* (2022).
- [47] Georgios Tsimos, Julian Loss, and Charalampos Papamanthou. 2022. Gossiping for communication-efficient broadcast. In *CRYPTO*.
- [48] Jiaping Wang and Hao Wang. 2019. Monoxide: Scale out blockchains with asynchronous consensus zones. In *NSDI*. 95–112.
- [49] Xin Wang, Haochen Wang, Haibin Zhang, and Sisi Duan. 2024. Pando: Extremely Scalable BFT Based on Committee Sampling. *Cryptology ePrint Archive* (2024).
- [50] Yibin Xu, Jingyi Zheng, Boris Döder, Tijs Slaats, and Yongluan Zhou. 2024. A Two-Layer Blockchain Sharding Protocol Leveraging Safety and Liveness for Enhanced Performance. In *NDSS*.
- [51] Jian Yin, Jean-Philippe Martin, Arun Venkataramani, Lorenzo Alvisi, and Mike Dahlin. 2003. Separating agreement from execution for byzantine fault tolerant services. *ACM SIGOPS Operating Systems Review* 37, 5 (2003), 253–267.
- [52] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT consensus with linearity and responsiveness. In *PODC*.
- [53] Haifeng Yu, Ivica Nikolić, Ruomu Hou, and Prateek Saxena. 2020. Ohie: Blockchain scaling made simple. In *S&P*. 90–105.
- [54] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. RapidChain: A Fast Blockchain Protocol via Full Sharding. In *CCS*. 931–948.
- [55] Haibin Zhang and Sisi Duan. 2022. PACE: Fully Parallelizable BFT from Reproducible Byzantine Agreement. In *CCS*.
- [56] Jianting Zhang, Zhongtang Luo, Raghavendra Ramesh, and Aniket Kate. 2024. Optimal Sharding for Scalable Blockchains with Deconstructed SMR. *arXiv preprint arXiv:2406.08252* (2024).

A ANALYSIS OF THE PROTOCOLS

A.1 Complexity Analysis of Existing Protocols

We analyze the communication complexities of the protocols mentioned in Table 1. We consider the shard size to be λ for all the protocols. We consider that a certificate with $O(\lambda)$ signatures is instantiated by digital signatures, so the size of the certificate is $O(\kappa\lambda)$.

GearBox [20]. The idea of GearBox is to design a protocol that carefully chooses the quorum size at the shard level to ensure that safety violation occurs with extremely low probability, even with small shard sizes. At the shard level, the protocol is similar to that of HotStuff [52] and PBFT [16] and we omit the details here.

GearBox uses a $\Pi_{\text{BD-TL}}$ protocol as a *control chain* to receive the "heartbeat" messages from all replicas in the system. $\Pi_{\text{BD-TL}}$ is executed by all n replicas. Informally, $\Pi_{\text{BD-TL}}$ is a standalone ledger that can be instantiated by conventional BFT (i.e., ABC) protocols [16, 52].

To monitor each shard, a committee is sampled. The committee is expected to be much larger than the shard but has $O(\lambda)$ size. Here, we consider the committee a "coordinating shard".

The protocol roughly works as follows. We highlight the workflow for each shard in blue for the purpose of shard coordination. Workflow in black is executed by all the committees or the entire system (for the control chain).

- (Get) Each replica P_i in shard S sends a heartbeat message (i.e., a timestamp of $O(1)$ size) to $\Pi_{\text{BD-TL}}$.
- (Check) Each replica P_j in the committee listens to the result on $\Pi_{\text{BD-TL}}$. After more than half of replicas in S submit their timestamps, P_j sends a request to S to obtain the latest "commit history" (we consider the length to be L) together with the proof (i.e., the certificate). Then, P_j sends the hash of the commit history and the proof to $\Pi_{\text{BD-TL}}$.
- (Reconfigure) Based on the messages submitted to $\Pi_{\text{BD-TL}}$ by the committee members, each replica P_i verifies whether S should be re-shuffled. If so, P_i submits a request to $\Pi_{\text{BD-TL}}$. We omit the details here.

THEOREM A.1. *The communication complexity of each shard is $O(L\lambda + \kappa\lambda^2)$ for the normal-case operation and $O(n)$ for shard management.*

PROOF. For the normal case, the communication complexity is:

$$\underbrace{O(L\lambda)}_{\text{propose}} + \underbrace{\sum_{i=1}^{\lambda} O(\kappa\lambda)}_{\text{vote}} + \underbrace{O(\kappa\lambda^2)}_{\text{collect}} = O(L\lambda + \kappa\lambda^2) \quad (1)$$

For the shard management of each shard S , the communication complexity (the procedures highlighted in blue) is $O(n)$, as every replica sends a heartbeat message to $\Pi_{\text{BD-TL}}$. $\square$

THEOREM A.2. *The communication complexity of the coordinating shard (i.e., the committee) is $O(L\lambda + \kappa\lambda^2)$.*

Proof. Each committee member listens to $\Pi_{\text{BD-TL}}$ and then performs the (Check) procedure. The communication complexity is thus:

$$\underbrace{\sum_{i=1}^{\lambda} O(L\lambda)}_{\text{query commit history}} + \underbrace{\sum_{i=1}^{\lambda} O(\kappa\lambda^2)}_{\text{query the proof}} + \underbrace{O(\kappa n)}_{\text{submit to } \Pi_{\text{BD-TL}}} \quad (2)$$

$$= O(L\lambda^2 + \kappa\lambda^3 + \kappa n)$$

Reticulum [50]. The motivation of Reticulum is to use a concretely small shard size to process and execute the requests. If all replicas in the shard S are correct, the block of requests can be *a-delivered* immediately. In other cases, a *control shard* with a bigger size will help manage the shard.

Reticulum considers a synchronous network and the protocol roughly proceeds as follows for each shard S .

- (Propose) The leader sends a proposed block of length L to all replicas in S .
- (Vote) Upon receiving the proposal, each replica in a shard S queries the $(\Delta + \delta)$ -BB protocol [4] to disseminate its vote (length $O(\kappa)$) to all replicas in S and the control shard.
- (Decide) If all replicas in S vote for the block, the block is immediately *a-delivered*. Otherwise, the control shard makes the decision. If not all replicas in S vote for the block, each replica may need to send the entire state and the block to the control shard.

The major bottleneck of the protocol is the "vote" phase. As there are λ replicas in S , λ instances are queried. Reticulum proposes to use a combined $(\Delta + \delta)$ -BB protocol, where several phases are combined to lower the communication complexity. We omit the discussion about the technical details. Using digital signatures as the certificate, the combined $(\Delta + \delta)$ -BB protocol achieves $O(\kappa\lambda^3)$ communication. We consider this to be the cost of shard management.

To conclude, the communication complexity of each shard is $O(L\lambda + \kappa\lambda^3)$, where $O(\kappa\lambda^3)$ is the cost for shard management. The communication complexity of the control shard (i.e., coordinating shard in our notion) is $O(\kappa\lambda^3 + L\lambda)$.

CoChain [36]. CoChain uses the concept of *Consensus on Consensus (CoC)* to handle faulty shards. Namely, the idea is to use a group of shards to agree on the order of both intra-shard and cross-shard transactions. The idea shares some similarity with our work. We consider the shards that are not the entry shard as the coordinating shards. For each entry shard S , a group of m shards are selected as the coordinating shards. The protocol is similar to a shard-level PBFT protocol where each shard is considered a "replica". Informally, each replica in a shard need to collect $2m/3$ matching messages (where each message is a certificate provided by a shard) before proceeding to the next phase.

For the communication between shards, i.e., shard A sends a message to shard B , we assume all nodes in shard A send a message to all nodes in shard B . Additionally, CoChain claims that each shard executes a BFT protocol. We thus use C_λ^L to denote the communication complexity.

Before analyzing the complexity, we also need to take a closer look at m . While the protocol does not specify the size of m , we argue that m has to be at least $O(\kappa)$ size. To see why, CoChain designs CoC to handle the case where S is faulty. As PBFT requires two-thirds of "replicas" to be correct, at least $2m/3$ shards must be correct. The replicas are randomly assigned to the shards, so the probability of each shard being faulty is uniform. If m is a constant, it obviously does not hold that at least $2m/3$ shards are correct. Therefore, we consider $m = O(\kappa)$.

THEOREM A.3. *The communication complexity of each shard is $O(C_\lambda^L)$ for the normal-case operation and $O(\kappa^3\lambda^2)$ for shard management. The communication complexity for the coordinating shard(s) is $O(\kappa^3\lambda^2)$.*

PROOF. The communication complexity of the protocol is:

$$\begin{aligned}
& \underbrace{C_\lambda^L}_{\text{entry}} + \underbrace{\sum_{i=1}^m \sum_{j=1}^\lambda O(\kappa\lambda^2)}_{\text{pre-prepare}} + \underbrace{\sum_{i=1}^m \sum_{j=1}^\lambda O(\kappa m\lambda) + O(mC_\lambda^K)}_{\text{prepare, commit}} \\
& + \underbrace{\sum_{i=1}^m \sum_{j=1}^\lambda O(\kappa\lambda) + O(mC_\lambda^K)}_{\text{finalize}} = O(C_\lambda^L + \kappa^2\lambda^3 + \kappa^3\lambda^2 + C_\lambda^K)
\end{aligned} \tag{3}$$

For each shard, the communication is $O(C_\lambda^L)$ for the normal-case operation and $O(\kappa^2\lambda^3)$ for shard management. Also, the communication complexity for the coordinating shards is $O(C_\lambda^K + \kappa^2\lambda^3 + \kappa^3\lambda^2)$. $\square$

A.2 Complexity Analysis of Otter

Complexity of the normal-case protocol. The normal-case protocol for each shard S_e involves the communication of replicas from at most two shards at a time. Let the shard size be λ . The message complexity of the normal-case protocol is $O(\lambda^2)$. We now discuss the communication complexity. In the first phase, the leader in S_e sends its proposal and a QC to all replicas in S_e . In other phases, replicas only need to exchange the hash of the block. We can instantiate the QCs using either aggregate signatures or threshold signatures. The normal-case protocol achieves $O(L\lambda + \kappa\lambda^2)$ communication with either instantiation. Meanwhile, the communication complexity for each coordinating shard is $O(\kappa\lambda^2)$.

Complexity of the rotating leader protocol. The message complexity of the rotating leader protocol is $O(\lambda)$ as one replica interacts with all replicas in a shard at a time. The communication complexity for the shard S_e is summarized below. We consider the procedure of communicating with the coordinating shard to be the cost for shard management and highlight the complexity in blue. Each QC consists of $|S_e|$ digital signatures with a total length of $O(\kappa\lambda)$. Meanwhile, the communication cost for the coordinating shard is $O(\kappa\lambda)$.

$$\sum_{i=1}^\lambda \underbrace{O(\kappa\lambda)}_{\text{query}} + \underbrace{O(\kappa\lambda)}_{\text{forward}} + \underbrace{O(\kappa\lambda)}_{\text{state}} + \underbrace{O(\kappa\lambda^3)}_{\text{new-epoch}} = O(\kappa\lambda^3) \tag{4}$$

The length of each QC can be optimized using an aggregate signature with length $O(\kappa \log \lambda)$. The communication complexity is then $O(\kappa\lambda^2 \log \lambda)$.

B CROSS-SHARD TRANSACTIONS

Extending MC-ABC to handle cross-shard transactions. We can extend the MC-ABC primitive to handle cross-shard transactions. In particular, there are two types of transactions: intra-shard transactions and cross-shard transactions. An *intra-shard* transaction needs to be processed by only one shard. A *cross-shard* transaction needs to be processed by more than one shard.

Without loss of generality, we consider cross-shard transactions that involve two shards at a time. The two shards are called *source*

shard and *target* shard, respectively. Let S_e and S_t be the source shard and target shard, respectively.

For any cross-shard transaction tx , universal safety should still be achieved. In the context of cross-shard transactions, universal safety focuses on the guarantee of the target shard only, i.e., total order of transactions is achieved at the source shard, but no guarantee is provided for the target shard. Such a guarantee might be sufficient for some cross-shard transactions. For instance, consider a transfer transaction where Alice transfers 100 tokens to Bob, where Alice's account x is maintained by S_e and Bob's account y is maintained by S_t . The transaction can be split into two sub-transactions: $x \leftarrow x - 100$, $y \leftarrow y + 100$. If all cross-shard transactions are transfer transactions, we only need to ensure the total order for S_e , i.e., correct replicas in S_e execute cross-shard transactions in the same order but replicas in S_t do not have to. Briefly speaking, this is because the target shard only executes the addition operations, the execution order does not matter. However, for other types of transactions, we may also need to further ensure that correct replicas in S_t also execute the transactions in the same order. We thus need a stronger safety guarantee than universal safety. Additionally, we also need to consider liveness guarantees.

Namely, besides universal safety, intra-shard liveness, common prefix, and bounded forks, an MC-ABC protocol should achieve the following property with probability $1 - \text{negl}(\kappa)$.

- **Universal cross-shard safety.** Let S_e be the source shard and S_t be the target shard for cross-shard transactions tx and tx' . If S_e and S_t are correct shards and a correct replica in $S_e \cup S_t$ *a-delivers* tx before *a-delivering* tx , then no correct replica in $S_e \cup S_t$ *a-delivers* tx' without first *a-delivering* tx .
- **Cross-shard liveness.** Let S_e be the source shard and S_t be the target shard for a cross-transaction tx . If S_e and S_t are correct shards and a correct replica in $S_e \cup S_t$ *a-broadcasts* tx , then all correct replicas in $S_e \cup S_t$ *a-deliver* tx .

The protocol. We extend our normal-case protocol to handle cross-shard transactions, as shown in Algorithm 6. In particular, the protocol proceeds as follows. First, replicas in S_e execute the "Propose" phase as in the normal-case protocol. Specifically, the leader proposes m , a batch of cross-shard transactions where S_e is the source shard and S_t is the target shard. The leader in S_e also sends the (PROPOSE) message to all replicas in S_t . Second, replicas in S_t treat the (PROPOSE) message as a transaction and add the message to their queue of transactions. Replicas in S_t then execute the "Co-Propose" phase by assigning an epoch number to m . After that, the "Commit" and "Finalize" phases are the same as our protocol for intra-shard transactions, i.e., two coordinating shards are sampled to agree on the order. We additionally require all replicas in S_t to execute the "Commit" and "Finalize" phases. Finally, after receiving a quorum of (FINALIZE) messages from $Q(S_{2,r})$ replicas in $S_{2,r}$, each replica in S_e and S_t *a-delivers* m .

Correctness and discussion. It is not too difficult to see that universal cross-shard safety and cross-shard liveness hold. Namely, compared to Algorithm 2, we only additionally require replicas in S_t to send (Co-Propose) messages. If both S_e and S_t are correct shards, safety can be ensured. For liveness, if the leader of S_t is correct, cross-shard liveness obviously holds. Otherwise, since all

Algorithm 6 The normal-case operation of Otter for cross-shard transactions. Pseudocode for replica P_i . S_e is the source shard and S_t is the target shard. Changes on top of Algorithm 2 are highlighted in gray.

```

1: initialization: let epoch  $r \leftarrow 1$ ,  $Q$  be the queue of pending transactions,
    $lr \leftarrow 1$ ,  $lockedQC \leftarrow \perp$ ,  $highQC \leftarrow \perp$ ,  $liveQC \leftarrow \perp$ ,  $LE \leftarrow \perp$ 
2:  $\triangleright$  Propose phase, executed by  $S_e$ 
3: upon  $a$ -broadcast( $m$ )
4:   if  $P_i$  is the leader for epoch  $r$  in  $S_e$  then
5:     obtain  $m$  from  $Q$ , where  $m$  is a batch of cross-shard transactions
6:     send (PROPOSE,  $e, r, m, highQC$ ) to all replicas in  $S_e$  and  $S_t$ 
7:  $\triangleright$  Co-propose phase, executed by  $S_t$ 
8: upon receiving (PROPOSE,  $e, r, m, highQC$ ) from the leader in  $S_e$  such
   that  $e > LE(S_e)$ 
9:   add the (PROPOSE) message to the queue
10:  if  $P_i$  is the leader in  $S_t$  then
11:    send (Co-PROPOSE,  $t, r', hash(m)$ ) to all replicas in  $S_e$  and  $S_t$ ,
    where  $r'$  is the epoch number of  $S_t$ 
12:  upon receiving a valid (Co-PROPOSE,  $t, r', hash(m)$ ) from the
    leader in  $S_t$ 
13:    send (Co-PROPOSE,  $t, r', hash(m)$ ) to all replicas in  $S_e$ 
14:  $\triangleright$  Prepare phase at  $S_e$ , executed by  $S_e$ 
15: upon receiving (PROPOSE,  $e, r, m, highQC$ ) from the leader  $P_j$  s.t.
     $m$  is not conflicting with  $lockedQC$  and  $highQC$  is the highest
     $liveQC$  in the  $M$  field of the (NEW-EPOCH) message (see Algorithm 5),
    and  $Q(S_t)$  matching (Co-PROPOSE,  $t, r, hash(m)$ ) messages from  $S_t$ 
16:    $S_{1,r}, \sigma_1 \leftarrow \text{Coin}(|r||1)$ 
17:   Let  $CoP$  be the  $Q(S_t)$  (Co-PROPOSE) messages
18:   send (PREPARE,  $e, r, hash(m), CoP, \sigma_1$ ) to  $S_{1,r}$  and  $S_e$ 
19:  $\triangleright$  Commit and Finalize phases, executed by  $S_{1,r}, S_{2,r}, S_e$ , and  $S_t$ 
20: Execute the "Commit" phase and "Finalize" phases are the same as
    Algorithm 2, besides that all messages are also sent to  $S_t$ 
21:  $\triangleright$  Executed by  $S_e$  and  $S_t$ 
22: upon receiving (FINALIZE,  $e, r, hash(m), \sigma_2, qc$ ) from  $P_j$  and  $P_j \in S_e$ 
    or  $\text{Coord}(j, e, 2, r)$  returns true based on  $\sigma_2$ 
23:   wait for  $Q(S_{2,r})$  (FINALIZE,  $e, r, hash(m), \sigma_2, qc$ ) messages from
     $S_{2,r}$  and  $Q(S_e)$  messages from  $S_e$ 
24:    $a$ -deliver( $m$ ) and  $m$ 's ancestors
25:   if  $m$  or  $m$ 's ancestors consist of cross-shard transactions then
26:     forward the (FINALIZE) messages to  $S_t$ 
27:   enter epoch  $lr + 1$ 

```

correct replicas add the (PROPOSE) message to their queue (line 9), a correct leader will be selected.

Note that we do not discuss or provide any security properties when either S_e or S_t is a faulty shard. In fact, when S_e is a faulty shard, there is no guarantee on safety or liveness for a cross-shard transaction tx . An easy explanation about the fact is that faulty replicas in S_e can simply keep silent so tx will never be a -delivered. For safety, if a correct replica P_i in S_e a -delivers tx with epoch r and another correct replica P_j in S_e a -delivers tx' in epoch r , our common prefix and bounded forks can only ensure that one of P_i or P_j will eventually abort. In such a scenario, if P_i a -delivers tx

and S_t is a correct shard, correct replicas in S_t will also a -deliver tx following our protocol. However, if P_i aborts later, our current protocol does not take any actions for replicas in S_t . We believe safety and liveness guarantees can be guaranteed via a robust recovery protocol for replicas that abort to become active again, as discussed in Sec. 7.

C INSTANTIATING THE $\text{Coin}()$ FUNCTION

Threshold PRF. Threshold PRF was first proposed by Cachin, Kursawe, and Shoup (CKS) [15]. An instantiation is provided, which is secure under the computational Diffie-Hellman (CDH) assumption in the random oracle model. Consider n replicas in the system, among which at most f are faulty. The replicas hold shares of an unpredictable function F that maps a value C (an arbitrary bit string) to its value $F(C) \in \{0, 1\}^K$. A threshold PRF scheme has four algorithms: $tgen$, $tsgn$, $shareverify$, and $tcombine$. To generate a threshold PRF value, all replicas have access to a common input C .

- **Unpredictability.** Before at least one correct replica releases the threshold PRF value, the probability that the adversary predicts any bit of the PRF value is $1/2 + \text{negl}(\kappa)$, where $\text{negl}(\kappa)$ is a negligible function.
- **Robustness.** It is computationally infeasible for an adversary to produce a PRF value C and $t + 1$ valid shares of C such that the output of the share combining algorithm is not $F(C)$.

Discussion. The unpredictability and robustness properties hold when there are no more than f faulty replicas.

When there are more than f faulty replicas, the unpredictability property does not hold anymore. Luckily, we do not need the unpredictability property. Namely, we consider a static adversary model. Even if the adversary predicts the identity of the randomly sampled shard, the adversary cannot corrupt the shard and make it faulty. We can thus still use the same probability analysis to analyze the correctness of our protocol.

The robustness property still holds even if there are more than f faulty replicas. In the context of the common coin, robustness means that the value of the coin cannot be *manipulated* by the adversary. In the CKS scheme, the correctness of robustness follows from the soundness of the interactive proof of equality of discrete logarithms and the random oracle assumption. Informally, this is because the $tgen$ algorithm is executed by a trusted dealer and the value of C is *fixed* in every epoch, and every correct replica can verify the share, even if the share is generated by an adversary. We refer the readers to [15] for more details.

Optimization. Our $\text{Coin}()$ function generates the identifier of a randomly selected shard together with a proof. In our optimization, we require that all shards are aware of the verification keys of all other shards. For example, given a shard S_e , suppose $(sk_e, vk_e) \leftarrow tgen()$. We assume all replicas in the system obtain vk_e from the dealer. Here, the threshold of the threshold PRF scheme is set as $|WQ(S_e)| - 1$.

Given a shard S_e , the $\text{Coin}(C)$ function generates v, σ , where v is the coin and σ is the proof. In our case, v is generated as follows. First, replicas query the threshold PRF function and generate $prf \leftarrow F(C)$ as mentioned above. Second, each replica calculates v as $v \leftarrow$

$\text{hash}(prf) \bmod s + 1$, where s is the total number of shards in the system. Finally, σ consists of C and $\mathcal{WQ}(S_e)$ shares. After receiving the shares, any replica first combines the shares by querying the $t\text{combine}$ algorithm and then obtains prf' . It then verifies whether $v \leftarrow \text{hash}(prf') \bmod s + 1$.

D PROOF OF CORRECTNESS

Without loss of generality, we assume each proposal consists of only one transaction.

LEMMA D.1. *Let $\alpha = 1/3 - \epsilon$ be the fraction of faulty replicas in the system and ϵ is a small constant where $0 < \epsilon \leq 1/3$, δ be the desired failure probability. If the number of the replicas in the committee is greater than $\frac{3\alpha}{\epsilon^2} \ln \frac{1}{\delta} = O(\lambda)$, then with probability $1 - \text{negl}(\kappa)$, the number of faulty replicas in the committee is less than $t = \lambda/3$ and the number of correct replicas in the committee is more than $2\lambda/3$. Here λ is a security parameter which can be considered as $O(\kappa)$.*

PROOF. Correctness follows [49, Lemma 1]. $\square$

LEMMA D.2. *Let $\alpha = \frac{1}{3} - \epsilon$ be the fraction of faulty replicas in the entire system, δ be the desired failure probability of each shard, and ϵ be a small constant where $0 < \epsilon < \frac{1}{3}$. If the number of the replicas in each shard S is greater than $\frac{3\alpha}{\epsilon^2} \ln \frac{1}{\delta} = O(\lambda)$, then with probability $1 - \delta \frac{1-\epsilon^2}{\epsilon^2}$, there exist at least $\mathcal{WQ}(S)$ correct replicas in shard S .*

PROOF. Correctness follows [49, Lemma 4]. $\delta \frac{1-\epsilon^2}{\epsilon^2}$ is a negligible function. $\square$

LEMMA D.3. *Assuming that the probability of each shard being faulty is δ and let S_e be the entry shard for intra-shard transactions tx and tx' . If a correct replica P_i a-delivers a transaction tx in epoch r and another correct replica P_j a-delivers a transaction tx' in epoch r , $tx = tx'$ with probability of at least $O(1 - \delta^2)$.*

PROOF. If P_i a-delivers tx in epoch r , it has received $Q(S_{2,r})$ matching (FINALIZE) messages from $S_{2,r}$ for tx . Similarly, if P_j a-delivers tx' , it has received $Q(S_{2,r})$ matching (FINALIZE) messages from $S_{2,r}$ for tx' . If $tx \neq tx'$, there are two cases: 1) $S_{2,r}$ is a faulty shard and $S_{2,r}$ has more than $Q(S_{2,r})$ faulty replicas; 2) There exist two correct replicas P_ℓ and P_k in $S_{2,r}$ such that P_ℓ receives $Q(S_{1,r})$ (COMMIT) messages from replicas in $S_{1,r}$ for tx and P_k receives $Q(S_{1,r})$ (COMMIT) messages from replicas in $S_{1,r}$ for tx' .

Case-1. The adversary can already make $tx = tx'$. The probability is $\Pr(\text{Case-1}) = \delta \frac{1-\epsilon^2}{\epsilon^2}$ according to Lemma D.2.

Case-2. If $S_{2,r}$ is a correct shard, it is impossible that $tx' \neq tx$ according to the quorum intersection rule. Therefore, case-2 can cause $tx \neq tx'$ only if $S_{2,r}$ is a faulty shard and the probability is $\Pr(S_{2,r} \text{ is faulty}) = \delta$.

If P_i receives $Q(S_{1,r})$ (COMMIT) messages from replicas in $S_{1,r}$ for tx and P_k receives $Q(S_{1,r})$ (COMMIT) messages from replicas in $S_{1,r}$ for tx' , there are two sub-cases: 1) $S_{1,r}$ is a faulty shard and $S_{1,r}$ has more than $Q(S_{1,r})$ faulty replicas; 2) $S_{1,r}$ is a correct shard and there exist two correct replicas P_1 and P_2 in $S_{1,r}$ such that P_1 receives $Q(S_e)$ messages from S_e for tx and P_2 receives $Q(S_e)$ messages from S_e for tx' .

Sub-case 1 occurs with probability of $\delta \frac{1-\epsilon^2}{\epsilon^2}$ according to Lemma D.2. Sub-case 2 occurs when S_e is a faulty shard (as otherwise the quorum intersection rule is violated), the probability of which is δ . Since the transactions are already allocated to the entry shards, we do not need to consider the probability of whether S_e is faulty. Therefore, the probability of sub-case-2 is the probability that $S_{1,r}$ is a faulty shard, i.e., δ .

$$\begin{aligned} \Pr(\text{Case-2}) &= \Pr(S_{2,r} \text{ is faulty}) \times (\Pr(S_{1,r} \text{ has more than } \\ &\quad Q(S_{1,r}) \text{ faulty replicas}) + \Pr(S_e \text{ is faulty})) \\ &= \delta \left(\delta \frac{1-\epsilon^2}{\epsilon^2} + \delta \right) < 2\delta^2 = O(\delta^2) \end{aligned} \quad (5)$$

The probability that $tx \neq tx'$ is therefore:

$$\begin{aligned} \Pr(\text{Case-1}) + \Pr(\text{Case-2}) &= \delta \frac{1-\epsilon^2}{\epsilon^2} + \delta \left(\delta \frac{1-\epsilon^2}{\epsilon^2} + \delta \right) \\ &= O(\delta^2) \end{aligned} \quad (6)$$

This completes the proof of the lemma. $\square$

LEMMA D.4. *Let S_e be the entry shard for an intra-shard transaction tx and let S_e be a correct shard. If a correct replica $P_i \in S_e$ a-delivers tx in epoch r , any proposal by a replica in epoch $r' > r$ that is conflicting with tx will not be accepted by $\mathcal{WQ}(S_e)$ correct replica in S_e .*

PROOF. Any correct replica $P_j \in S_e$ in epoch $r' > r$ accepts a transaction conflicting with tx only if its *lockedQC* is not an rQC for tx or a transaction higher than r . According to the protocol, any correct replica $P_j \in S_e$ sets its *lockedQC* as a QC for transaction tx in epoch r after receiving (COMMIT) messages from at least $Q(S_{1,r})$ replicas in $S_{1,r}$. Meanwhile, if $P_i \in S_e$ a-delivers tx in epoch r , it has received (FINALIZE) messages from $Q(S_{2,r})$ replicas in $S_{2,r}$ and also $Q(S_e)$ replicas in S_e . Therefore, any correct replicas in $Q(S_e)$ set their *lockedQC* either as a QC for tx or a transaction tx' with a greater epoch number, and tx' extends tx . Since S_e is a correct shard, the number of correct replicas that will not accept a transaction that is conflicting with tx is $\mathcal{WQ}(S_e)$. $\square$

LEMMA D.5. *Let S_e be the entry shard and $S_{0,r}$ be the coordinating shard for epoch r . If $S_{0,r}$ is a correct shard and the leader in epoch r sends a (NEW-EPOCH, r, M, π) message to all replicas in S_e , any correct replicas in S_e receive the same (NEW-EPOCH, r, M, π) message.*

PROOF. Towards a contradiction, assume that a correct replica $P_i \in S_e$ receives (NEW-EPOCH, r, M, π) and another correct replica $P_j \in S_e$ receives (NEW-EPOCH, r', M', π') message. As π consists of $Q(S_{0,r})$ digital signatures for $\text{hash}(M)$ and π' consists of $Q(S_{0,r})$ digital signatures for $\text{hash}(M')$, either the collision resilience of the hash function is violated, or at least one correct replica P_j in $S_{0,r}$ sends both a (STATE, $r, \text{hash}(M), \pi_j$) message and a (STATE, $r, \text{hash}(M'), \pi_j$) message, a contradiction. $\square$

THEOREM D.6 (UNIVERSAL SAFETY). *If S_e is a correct shard and a correct replica a-delivers a transaction tx before a-delivering tx' , then no correct replica a-delivers tx' without first a-delivering tx with probability $1 - \text{negl}(\kappa)$.*

PROOF. We show that universal safety holds with probability $O(1 - \delta^2)$. Consider that a correct replica P_i *a-delivers* tx in epoch r_1 and *a-delivers* tx' in epoch r_2 and $r_1 < r_2$. Towards a contradiction, consider that a correct replica P_j *a-delivers* tx' in epoch r_3 and *a-delivers* tx in epoch r_4 and $r_3 < r_4$. Without loss of generality, we assume that $r_1 < r_3$, i.e., $r_1 < r_3 < r_4$.

According to Lemma D.3, if P_i *a-delivers* tx in epoch r and P_j *a-delivers* tx'' in epoch r , $tx = tx''$ with probability $O(1 - \delta^2)$. Since a correct replica will never *a-deliver* a transaction twice, the fact that P_j *a-delivers* tx in epoch r_4 is a contradiction with the assumption that $r_1 < r_4$. The theorem thus holds. $\square$

THEOREM D.7 (EXTENDED EVENTUAL FORK DETECTION). *Assuming that the probability of each shard being a faulty shard is δ and let S_e be the entry shard for transaction tx . Let S_e be a faulty shard and let the number of correct replicas in S_e be $\Omega(|S_e|)$. In some epoch r , if a correct replica P_i maintains a chain C_i , a correct replica P_j maintains a chain C_j and $C_i \xleftrightarrow{\text{fork}} C_j$, at most one of the replicas will extend its chain in each epoch r with probability $1 - \text{negl}(\kappa)$. Eventually, one of the replicas will abort.*

PROOF. Let the chain maintained by P_j be C_j and let $\text{head}(C_j)$ be tx . If P_j *a-delivers* tx , it has received $Q(S_e)$ matching (FINALIZE) messages for tx and epoch r . According to the protocol, at least $Q(S_e) - \mathcal{F}(S_e)$ correct replicas have previously received matching (COMMIT) messages from $Q(S_{1,r})$ replicas in $S_{1,r}$ and set their *lockedQC* as a *cQC* for tx before they send the (FINALIZE) messages. Meanwhile, at least $Q(S_e)$ replicas have sent a (COMMIT) message for tx and set up their *liveQC*[r] as the set of (COMMIT) messages. In the following, we use *lockqc* to denote the *lockedQC* for tx and I to denote the $Q(S_e)$ replicas that send the (FINALIZE) messages.

If P_i maintains C_i , at least $Q(S_e) - \mathcal{F}(S_e)$ correct replicas are locked on $\text{head}(C_i)$ or a block that extends C_i . According to the protocol, every correct replica only considers a proposal valid if the proposed block extends its locked block. Let epoch r^* be the epoch number such that after epoch r^* , the forks of C_i and C_j start. We let r_i be the epoch number of C_i and r_j be the epoch number of C_j . Without loss of generality, let $r_j \geq r_i$.

Let the current epoch number of the system be r . Obviously, $r \geq r_j$. During the epoch change in any epoch $r' > r$, there are three cases: 1) none of P_i or P_j receives a (NEW-EPOCH) message and resumes normal operation; 2) one of P_i and P_j receives a (NEW-EPOCH, r, M, π) message from the leader in epoch r' ; 3) both P_i and P_j receive the same (NEW-EPOCH, r, M, π) message from the leader in epoch r' .

In the first case, P_i and P_j will continue epoch changes until they resume normal-case operation. Neither of the two replicas extends its chain. In the second case, at most one of P_i and P_j extends its chain.

We now discuss the third case. First, with probability $1 - \delta$, $S_{0,r'}$ is a correct shard. According to Lemma D.5, P_i and P_j receive the same (NEW-EPOCH, r, M, π) message.

Second, we show that at most one of P_i and P_j extends its chain if $S_{0,r'}$ is a correct shard. Assume that P_j extends its chain and resumes normal-case operation. According to our protocol, there does not exist any *lqc* in the (QUERY) messages in M such that *lqc* is

conflicting with *lockqc* and the epoch number of the highest QC *qc* is at least the same as that of *lockqc*. It is then not very difficult to see that *qc* is a QC for a block that is either $\text{head}(C_j)$ or a block that extends $\text{head}(C_j)$. In this case, after P_i receives (NEW-EPOCH, r, M, π), the condition that there exists at least one *lqc* that is conflicting with P_i 's *lockedQC* holds. P_i aborts, so the theorem holds.

To conclude, the first case and the second case happen when the leader of epoch r' is faulty. As we assume that the system has at least $O(|S_e|)$ correct replicas, the probability that the third case happens is a constant. In the third case, the theorem is violated when $S_{0,r'}$ is a faulty shard. The theorem holds with probability $1 - \text{negl}(\kappa)$. $\square$

THEOREM D.8 (COMMON PREFIX). *Let S_e be a shard with more than $\frac{|S_e|-1}{3}$ faulty replicas and let the number of correct replicas in S_e be $\Omega(|S_e|)$. At epoch r , an active correct replica P_i maintains a chain C_i and an active correct replica P_j maintains a chain C_j , then $C_i^{[k]} \preceq C_j$ and $C_j^{[k]} \preceq C_i$ for a constant k .*

PROOF. According to Theorem D.7, if $C_i \xleftrightarrow{\text{fork}} C_j$ holds, at least one of P_i and P_j aborts with probability $1 - \delta$ in each epoch. Consider that after epoch r^* , the forks are formed. The probability that after epoch $r^* + k$, one of P_i and P_j does not abort is bounded by $1 - \delta^{\frac{c}{|S_e|}k}$, where c is the number of correct replicas in S_e . Since δ is $\text{negl}(\kappa)$, the probability $1 - \delta^{\frac{c}{|S_e|}k}$ is $1 - \text{negl}(\kappa)$. Therefore, k is a constant. $\square$

THEOREM D.9 (BOUNDED FORKS). *Let S_e be a shard with more than $\frac{|S_e|-1}{3}$ faulty replicas and $Q(S_e) - \mathcal{F}(S_e) = O(|S_e|)$. Let I be a set of replicas such that for any $P_i, P_j \in I$, $C_i \xleftrightarrow{\text{fork}} C_j$. The size of I is bounded by a constant, i.e., $|I| = O(1)$.*

PROOF. According to the protocol, every correct replica accepts a proposal if the proposed block extends its *lockedQC*. If one correct replica P_i *a-delivers* a transaction tx , at least $Q(S_e) - \mathcal{F}(S_e)$ correct replicas must have voted for tx in the normal-case operation. Let γ be the size of I . As we assume $Q(S_e) - \mathcal{F}(S_e) = O(|S_e|)$, γ is bounded by:

$$\gamma = \frac{|S_e|}{(Q(S_e) - \mathcal{F}(S_e))} = O(1). \quad (7)$$

$\square$

THEOREM D.10 (INTRA-SHARD LIVENESS). *Given that S_e has more than $Q(S_e)$ correct replicas and if a correct replica *a-broadcasts* an intra-shard transaction tx , then all correct replicas eventually *a-deliver* tx .*

PROOF. We assume that the timeout value for the rotating leader protocol is properly set up. As S_e participates in every phase of the protocol, every correct replica in S_i is able to obtain a *liveQC* if the leader in an epoch r is correct. Assume that tx is included in the proposal in epoch r . It is not difficult to see that if both $S_{1,r}$ and $S_{2,r}$ are correct shards, the probability of which is $O(1 - \delta^2)$. If one of $S_{1,r}$ and $S_{2,r}$ is faulty, a *liveQC* is formed among S_e . If tx is not *a-delivered* in epoch r , any correct replica sends its highest *liveQC* to $S_{0,r}$ and waits until it receives a *confirmQC* from $Q(S_{0,r})$ replicas. If $S_{0,r}$ is correct, M consists of the *liveQC* for tx and any correct replicas will consider a transaction tx' valid in epoch $r' > r$

if tx' extends tx . The probability that $S_{0,r}$ is a correct shard is δ . If the *liveQC* is not included in M , any correct leader will continue to include tx in its proposal m according to our protocol. Therefore, tx will eventually be *a-delivered*. $\square$